nastexo farax mahamed
id C6240388

nasteha@ubuntu:~$ man linux

PRINCIPLES OF LINUX OPERATING SYSTEMS

Complete Command Reference and Lab Guide

Chapters 1 - 4

Computer Science / Information Technology

Linux Operating System course - Student reference

Chapter 1: Introduction to Linux and Ubuntu
Chapter 2: Linux System Architecture, Command Line, and File Management
Chapter 3: Text Processing, Searching, Archiving, and Editors
Chapter 4: Process, Service, and System Management

Prepared by: nasteha
Terminal user / host: nasteha@ubuntu
Environment: Ubuntu 24.04 LTS, Bash
Date: 11 November 2026

Chapter 1: Introduction to Linux and Ubuntu

Chapter 1 overview - commands in this chapter

Command / topic	Section	No.	Output
whoami	Lab: Installing Ubuntu and Opening a Terminal	1.10.1	Real
pwd	Lab: Installing Ubuntu and Opening a Terminal	1.10.2	Real
ls	Lab: Installing Ubuntu and Opening a Terminal	1.10.3	Real
cat	Lab: Installing Ubuntu and Opening a Terminal	1.10.4	Real
uname	Lab: Installing Ubuntu and Opening a Terminal	1.10.5	Real
apt	Lab: Installing Ubuntu and Opening a Terminal	1.10.6	Real
dpkg	Lab: Installing Ubuntu and Opening a Terminal	1.10.7	Real
dnf	Lab: Installing Ubuntu and Opening a Terminal	1.10.8	Real
yum	Lab: Installing Ubuntu and Opening a Terminal	1.10.9	Real
zypper	Lab: Installing Ubuntu and Opening a Terminal	1.10.10	Real
Ctrl + Alt + T	Lab: Installing Ubuntu and Opening a Terminal	1.10.11	Representative

1.1 What Is an Operating System?

The operating system (OS) is the manager of the computer. Programs never talk to the disk or keyboard on their own; they ask the OS.

- Five jobs of the OS: shares the CPU, manages memory (RAM), manages files, controls hardware through drivers, provides a user interface (GUI or command line).
- Four types of OS: Desktop (Windows, macOS, Ubuntu), Server (Windows Server, Ubuntu Server, CentOS/Rocky), Mobile (Android, iOS), Embedded (routers, TVs, cars - often a small Linux).
- If the OS is not running, the computer is just hardware sitting there.
- Linux is free to use and its source code is open.

1.2 What Is Linux?

Linux has two meanings: strictly, the kernel begun by Linus Torvalds in 1991; in everyday speech, a complete system (a distribution) built around that kernel.

- Kernel, distribution and desktop are not the same thing.
- Analogy: Kernel = engine of a car; Distribution = complete car; Desktop = dashboard and controls.
- The GNU Project (1983) supplied free tools: compiler, shell and basic commands. Debian GNU/Linux names both parts.

- Linux uses Unix principles but contains no Unix code; an international community develops it.

Term	What it is	Examples	How you meet it in the lab
Kernel	Core that talks to the hardware; starts processes, gives memory, takes CPU back	The Linux kernel (1991)	uname -r shows its release
Distribution	Kernel + installer + programs + a way to install more software	Ubuntu, Debian, Fedora, Mint	Install an Ubuntu LTS Desktop ISO
Desktop	Graphical environment; optional (Ubuntu Server has none)	GNOME, KDE	Activities > type terminal

1.3 A Short History of Linux

Unix ideas, GNU tools, the Linux kernel, then Debian and Ubuntu on top.

- 1960s-70s: Unix at Bell Labs - files, processes, pipes, users and permissions.
- 1983: GNU Project (Richard Stallman) - free tools and the GPL licence.
- 1991: Linux kernel by Linus Torvalds, a student in Finland. 1992: kernel 0.12 first released under GPL v2.
- 1993: Debian (Ian Murdock). 2004: Ubuntu (Mark Shuttleworth and team), based on Debian and easier to install - our lab system.

1.4 The Linux Kernel and System Architecture

The kernel sits between programs and hardware and looks after six areas.

- Hardware, Processes, Memory, Files, Networking, Security.
- Layers from the bottom: Hardware > Kernel > Shell and small tools (Bash, ls, cp, mv) > Applications > Desktop (optional) > You, the user.
- A lot of things look like files, including devices; /proc and /sys are read later in the course.
- Ubuntu Server has no desktop: you log in with a terminal, often over the network.

1.5 Linux Distributions and Families

Each family uses its own package tools: same kernel, different toolbox.

- Enterprise grade: older, well-tested software, long support (RHEL: 10 years). Consumer grade: latest kernel and drivers, shorter updates. Experimental/hacker: newest software, rolling release (Arch).
- Run 'yum install...' = Red Hat language. On Ubuntu we use apt. Do not mix them on one machine.
- Kali Linux: penetration testing. QubesOS: secure desktop. CentOS: free rebuild of RHEL source.

Family	Packages	Tool	Distributions
Debian (our lab)	.deb	apt (dpkg underneath)	Debian, Ubuntu, Linux Mint, Raspbian
Red Hat	.rpm	dnf (older: yum)	Fedora, RHEL, CentOS, Rocky
SUSE	rpm	zypper (YaST)	openSUSE, SLES

1.6 Ubuntu Linux: Our Lab System

Ubuntu is Debian underneath with a friendlier installer and a regular release calendar.

- Ubuntu Desktop has GNOME; open the terminal from Activities or with Ctrl + Alt + T. Ubuntu Server has no desktop.
- A release every six months; a new LTS (Long Term Support) every two years. LPI: LTS = 5 years of security updates. Use LTS.
- Version numbers are YY.MM: 19.04 = April 2019. Download the ISO only from ubuntu.com.

1.7 Why Linux Matters

Linux is the right tool for servers, labs and engineering work, though not better at everything.

- Costs nothing to install; skills transfer; config files and logs are text.
- Runs web servers, cloud VMs (AWS, Google, Azure), Android phones, supercomputers, embedded/IoT devices, Raspberry Pi.
- Numbers: 100% of TOP500 supercomputers since 2017; ~90% of public cloud workload (2017); ~68% of website servers powered by Unix (mostly Linux).
- Hard parts: some Windows-only programs, some drivers (Nvidia, printers), the terminal feels strange at first.

1.8 Linux, Windows, and macOS Compared

We are not fighting Windows; we are learning the system you will meet at work.

- macOS is based on BSD Unix; uname -s reports Darwin. Linux is Unix-like with no Unix code.
- macOS and Linux feel closer on the command line than either does to old Windows cmd.

Criterion	Windows	macOS	Linux (Ubuntu)
Licence	Bought licence	Apple only	Free
Hardware	Most office PCs	Apple only	Many machines incl. old PCs
Heritage	Separate from Unix	BSD Unix	Unix-like
Interface	Bundled desktop	Bundled desktop	Choose GNOME, KDE…
Software	Installer: Next, Next	-	Package tools: apt, dnf
Command line	Old cmd	Terminal (zsh/Bash)	Terminal (Bash)
Typical use	Office PCs, games	Apple users	Servers, labs, engineering

1.9 Free and Open-Source Software and Licensing

'Free' means freedom, not no price.

- Four freedoms: 0 run the program; 1 study and change it (needs source); 2 share copies; 3 share your changes (needs source).
- Free software: FSF, Stallman, a social movement. Open source: OSI (1998, Eric S. Raymond and Bruce Perens), pragmatic; 80+ approved licences. FOSS/FLOSS joins both.
- Permissive: MIT, BSD, Apache, CC BY. Copyleft: GNU GPL (Linux kernel), AGPL. In between: LGPL, CC BY-SA.
- Do not copy code into a project until you know the licence. ownCloud: dual licensing (GPL + proprietary Business Edition).

1.10 Lab: Installing Ubuntu and Opening a Terminal

Run Ubuntu in a VirtualBox virtual machine (Route A), take a snapshot, open a terminal and run the first five commands.

- Roadmap: 1 Download VirtualBox + Ubuntu LTS ISO; 2 Create VM; 3 Install Ubuntu in the VM; 4 Snapshot week1-clean; 5 Open a terminal and run five commands.
- VM settings: Name Ubuntu; Type Linux / Ubuntu (64-bit); about 2 CPUs; 2048 MB RAM minimum (4096 MB if possible); VDI dynamically allocated disk ~40 GB; optical drive = Ubuntu ISO; first snapshot week1-clean.
- Installer: Install Ubuntu > language/keyboard > Erase disk and install (virtual disk only) > name and password > restart. If stuck on 'remove installation medium': Settings > Storage > remove ISO > Enter.
- Open the terminal: Activities > type terminal, or Ctrl + Alt + T. Prompt anatomy: username @ hostname : current directory ($ normal user, # root; ~ = home). We do not work as root.
- Week 1 rule: do not wipe your only Windows laptop.

Setting	Value for our lab
Name	Ubuntu (or your name + Ubuntu)
Type / Version	Linux / Ubuntu (64-bit)
Processors	About 2 CPUs
Memory	2048 MB minimum; 4096 MB if your PC can spare it
Virtual hard disk	VDI, dynamically allocated, around 40 GB
Optical drive	The Ubuntu ISO (Settings > Storage)
First snapshot	week1-clean

Command	What it shows	What you saw (evidence)
whoami	The name of the user running the shell	nasteha
pwd	The current (working) directory as an absolute path	/home/nasteha
ls	Contents of the current directory	(see Section 1.10 commands)

Command	What it shows	What you saw (evidence)
cat /etc/os-release	Contents of /etc/os-release	Ubuntu release details
uname -r	Release of the running Linux kernel	(kernel release)

1.10.1 whoami

Purpose: Prints the user name of the account running the shell.

Syntax

```
whoami [OPTION]
```

Options / Flags

Option / Flag	Meaning
--help	Display help and exit
--version	Output version information and exit

Example

```
$ whoami
$ whoami --version | head -n 1
```

- whoami prints the effective user name.
- Lab evidence sheet (Figure 1.31): first of the five commands.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ whoami
nasteha
nasteha@ubuntu:~$ whoami --version | head -n 1
whoami (GNU coreutils) 9.4
```

Screenshot: whoami in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.2 pwd

Purpose: Prints the current (working) directory as an absolute path.

Syntax

```
pwd [-L|-P]
```

Options / Flags

Option / Flag	Meaning
-L	Print the logical directory (may contain symlinks) - default
-P	Print the physical directory, symlinks resolved
--help	Display help

Example

```
$ pwd
```

```
$ cd /etc
$ pwd
$ cd
$ pwd
```

- pwd shows where you are; ~ in the prompt means your home directory.
- A new terminal opens in /home/<user>.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ pwd
/home/nasteha
nasteha@ubuntu:~$ cd /etc
nasteha@ubuntu:/etc$ pwd
/etc
nasteha@ubuntu:/etc$ cd
nasteha@ubuntu:~$ pwd
/home/nasteha
```

Screenshot: pwd in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.3 ls

Purpose: Lists the contents of the current directory (or of the directory given).

Syntax

```
ls [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
(none)	Names only
-a, --all	Include hidden names (starting with .)
-l	Long listing, one line per entry
-h, --human-readable	Sizes as K, M, G (with -l)
-R	Recurse into sub-directories
-1	One name per line
-d	List a directory itself, not its contents
-F	Append / to directories, * to executables
-t	Sort by modification time
-r	Reverse the sort order

Example

```
$ ls
$ mkdir Documents Downloads
$ touch notes.txt
$ ls
$ ls -l
```

- Plain ls in the home directory; the lab evidence sheet asks you to record what ls shows.

- Detailed use of -a, -l, -h continues in Section 2.9.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ ls
nasteha@ubuntu:~$ mkdir Documents Downloads
nasteha@ubuntu:~$ touch notes.txt
nasteha@ubuntu:~$ ls
Documents  Downloads  notes.txt
nasteha@ubuntu:~$ ls -l
total 8
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 Documents
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 Downloads
-rw-r--r-- 1 nasteha nasteha    0 Nov 11 18:43 notes.txt
```

Screenshot: ls in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.4 cat

Purpose: Concatenates files and prints them on standard output. Here it displays /etc/os-release.

Syntax

```
cat [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-n	Number all output lines
-b	Number non-empty lines only
-s	Squeeze repeated blank lines
-A	Show all characters ($ at line ends, ^I for tabs)
(no FILE)	Read standard input until Ctrl+D

Example

```
$ cat /etc/os-release
```

- cat /etc/os-release shows the distribution details; compare with uname -r: kernel and distribution are not the same thing.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
```

```
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

Screenshot: cat in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.5 uname

Purpose: Prints information about the running kernel and the system.

Syntax

```
uname [OPTION]...
```

Options / Flags

Option / Flag	Meaning
-r	Kernel release
-s	Kernel name (Linux on Linux, Darwin on macOS)
-n	Network node hostname
-m	Machine hardware (architecture)
-a	All fields on one line

Example

```
$ uname -r
$ uname -s
```

- uname -r prints the kernel release (slide example: 4.15.0-1019-aws).
- uname -s prints Linux here; macOS prints Darwin (Figure 1.18).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ uname -r
6.18.44-fc-v378
nasteha@ubuntu:~$ uname -s
Linux
```

Screenshot: uname in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

Note: The kernel release shown comes from the machine on which the document was produced; yours will differ.

1.10.6 apt

Purpose: Debian-family package tool (Ubuntu). Mentioned in Section 1.5: on Ubuntu we use apt, not yum.

Syntax

```
apt [OPTIONS] COMMAND
sudo apt install PACKAGE
```

Options / Flags

Option / Flag	Meaning
install	Install a package (needs sudo)
update	Refresh the package lists
upgrade	Upgrade installed packages
remove	Remove a package
search	Search package descriptions
--version	Show the apt version

Example

```
$ apt --version
```

- Only the version is shown here because installing needs administrator rights and network access (Chapter 6 topic).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ apt --version
apt 2.8.3 (amd64)
```

Screenshot: apt in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.7 dpkg

Purpose: Low-level tool of the Debian family that works with .deb packages (apt works on top of it).

Syntax

```
dpkg [OPTION]... ACTION
```

Options / Flags

Option / Flag	Meaning
-l	List installed packages
-i	Install a .deb file
-r	Remove a package
-s	Show package status
-L	List files of a package
--version	Show version

Example

```
$ dpkg --version | head -n 1
$ dpkg -l | head -n 5
```

- dpkg --version identifies the tool; dpkg -l lists installed packages (first lines shown).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ dpkg --version | head -n 1
Debian 'dpkg' package management program version 1.22.6 (amd64).
nasteha@ubuntu:~$ dpkg -l | head -n 5
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name                  Version                 Architecture Description
+++-=====================-=======================-============-
================================================================================
```

Screenshot: dpkg in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.8 dnf

Purpose: Package tool of the Red Hat family (Fedora, RHEL, Rocky) for .rpm packages. Not available on Ubuntu.

Syntax

```
dnf [OPTIONS] COMMAND PACKAGE
```

Options / Flags

Option / Flag	Meaning
install	Install a package
remove	Remove a package
update	Update packages
search	Search packages

Example

```
$ dnf install sl
```

- Typing dnf on Ubuntu fails: same kernel, different toolbox.

Terminal (screenshot of the command and its output)

```
● ● ● nasteha@ubuntu: ~
nasteha@ubuntu:~$ dnf install sl
bash: dnf: command not found
```

Screenshot: dnf in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.9 yum

Purpose: Older Red Hat-family package tool. If a website says 'run yum install...', that is Red Hat language.

Syntax

```
yum [OPTIONS] COMMAND PACKAGE
```

Options / Flags

Option / Flag	Meaning
install	Install a package

Option / Flag	Meaning
remove	Remove a package
update	Update packages

Example

```
$ yum install sl
```

- On Ubuntu use apt instead.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ yum install sl
bash: yum: command not found
```

Screenshot: yum in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.10 zypper

Purpose: Package tool of the SUSE family (openSUSE, SLES); YaST is its graphical companion.

Syntax

```
zypper [OPTIONS] COMMAND PACKAGE
```

Options / Flags

Option / Flag	Meaning
install	Install a package
remove	Remove a package
refresh	Refresh repositories
search	Search packages

Example

```
$ zypper install sl
```

- Not installed on Ubuntu.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ zypper install sl
bash: zypper: command not found
```

Screenshot: zypper in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

1.10.11 Ctrl + Alt + T

Purpose: Keyboard shortcut that opens a terminal on Ubuntu Desktop (alternative: Activities, then type terminal).

Syntax

```
Ctrl + Alt + T
```

Options / Flags

Option / Flag	Meaning
Ctrl	Hold
Alt	Hold
T	Press to open GNOME Terminal

Example

```
$ whoami
$ pwd
$ uname -r
```

- After the terminal opens, read the prompt before typing: user @ host : directory $.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ whoami
nasteha
nasteha@ubuntu:~$ pwd
/home/nasteha
nasteha@ubuntu:~$ uname -r
6.8.0-85-generic
```

Screenshot: Ctrl + Alt + T in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Not a command; shown as the entry point to the terminal. The sequence mirrors the title slide of Chapter 1.

Chapter 2: Linux System Architecture, Command Line, and File Management

Chapter 2 overview - commands in this chapter

Command / topic	Section	No.	Output
ls /home	Ubuntu System Architecture	2.1.1	Real
ls /	The Filesystem Hierarchy	2.2.1	Real
ls /sys/class/net	The Filesystem Hierarchy	2.2.2	Real
echo	The Terminal and Bash	2.3.1	Real
history	The Terminal and Bash	2.3.2	Representative
Tab completion	The Terminal and Bash	2.3.3	Representative
ls -l /home	Command Syntax	2.4.1	Real
man	Getting Help: man, --help, and info	2.5.1	Representative
--help	Getting Help: man, --help, and info	2.5.2	Real
info	Getting Help: man, --help, and info	2.5.3	Representative
help	Getting Help: man, --help, and info	2.5.4	Real
Quoting with " " and ' '	Quoting, Escaping, and Wildcards	2.6.1	Real
Escaping with a backslash	Quoting, Escaping, and Wildcards	2.6.2	Real
Wildcards * ? [ ]	Quoting, Escaping, and Wildcards	2.6.3	Real
Variables: NAME=value	Variables, PATH, and ~/.bashrc	2.7.1	Real
export	Variables, PATH, and ~/.bashrc	2.7.2	Real
echo $PATH	Variables, PATH, and ~/.bashrc	2.7.3	Real
~/.bashrc	Variables, PATH, and ~/.bashrc	2.7.4	Real
source	Variables, PATH, and ~/.bashrc	2.7.5	Real
alias (ll)	Variables, PATH, and ~/.bashrc	2.7.6	Real
cd with absolute and relative paths	Paths and Filenames	2.8.1	Real
Case sensitivity and hidden names	Paths and Filenames	2.8.2	Real
cd (pwd and cd together)	Navigating with pwd, cd, and ls	2.9.1	Real

Command / topic	Section	No.	Output
ls with -a, -l and -h	Navigating with pwd, cd, and ls	2.9.2	Real
touch	Creating and Managing Files	2.10.1	Real
mkdir	Creating and Managing Files	2.10.2	Real
rmdir	Creating and Managing Files	2.10.3	Real
cp	Creating and Managing Files	2.10.4	Real
mv	Creating and Managing Files	2.10.5	Real
rm	Creating and Managing Files	2.10.6	Real
sudo	Creating and Managing Files	2.10.7	Representative
Lab Part A (steps 1-7)	Lab: Navigating and Managing Files	2.11.1	Real
Lab Part B (steps 8-14)	Lab: Navigating and Managing Files	2.11.2	Real

2.1 Ubuntu System Architecture

A Linux system is built in layers; a command travels down the layers and the result travels back up.

- Six layers (top to bottom): Users, Applications (GNOME lives here, Ubuntu Desktop only), System utilities (ls, cp, mkdir in /usr/bin), Shell (Bash), Kernel, Hardware.
- Following 'ls /home': you press Enter; Bash splits the line and finds /usr/bin/ls; ls asks the kernel for the contents of /home; the kernel reads the disk through its driver; names travel back up and Bash shows a new prompt.
- The kernel shares CPU cores, hands RAM to programs, reaches disks through drivers (/dev/sda, /dev/nvme0n1) and devices through drivers + files in /dev.
- Virtualization: Host = real computer and its OS; Hypervisor (VirtualBox) creates VMs; Guest = Ubuntu in the VM with its own kernel on virtual hardware. The guest disk is a .vdi file on the host.
- The desktop (GNOME) is optional; the system still runs without it.

2.1.1 ls /home

Purpose: Lists the contents of /home; used in the slides to trace one command through the six layers.

Syntax

```
ls /home
```

Options / Flags

Option / Flag	Meaning
/home	Argument: the directory whose contents are listed

Example

```
$ ls /home
$ which ls
```

- Bash finds /usr/bin/ls, ls asks the kernel, the kernel reads the disk, the names come back up.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls /home
claude  nasteha  ubuntu
nasteha@ubuntu:~$ which ls
/usr/bin/ls
```

Screenshot: ls /home in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.2 The Filesystem Hierarchy

One tree from / - no drive letters. Even a USB stick appears somewhere under /.

- /proc and /sys are pseudo-filesystems built by the kernel in memory; they take no space on disk and vanish at shutdown. /proc: processes and kernel (cpuinfo, meminfo). /sys: devices and drivers (/sys/class/net).
- '/usr means user files' is wrong: /usr holds installed programs; your files are in /home. On Ubuntu /bin is a shortcut to /usr/bin.
- Where do I change the machine's name? /etc/hostname. Disk filling up? Look in /var. Back up everyone's work? Copy /home.

Directory	Holds
/home	One directory per user: /home/nasteha
/etc	System-wide configuration, mostly plain text
/var	Data that changes: logs, caches, queues
/usr	Installed programs and their files (/usr/bin)
/tmp	Temporary files; usually cleared at reboot
/boot	The kernel and files needed to start up
/dev	Device files: disks, terminals, USB
/proc	Live process and kernel information (memory only)
/sys	Live device and driver information (memory only)

2.2.1 ls /

Purpose: Lists the top-level directories of the filesystem tree.

Syntax

```
ls /
```

Options / Flags

Option / Flag	Meaning
/	The root directory, start of the single tree

Example

```
$ ls /
```

- Your output may show a few extra names (Figure 2.4 note).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls /
bin
boot
dev
etc
home
lib
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
```

Screenshot: ls / in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.2.2 ls /sys/class/net

Purpose: Lists network interfaces published by the kernel under /sys (kernel interface, not on disk).

Syntax

```
ls /sys/class/net
```

Options / Flags

Option / Flag	Meaning
/sys/class/net	One folder per network interface

Example

```
$ ls /sys/class/net
```

- In the slides the VM shows enp0s3 and lo; interface names vary by machine.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

Screenshot: ls /sys/class/net in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.3 The Terminal and Bash

The terminal window and the shell inside it are different programs. On Ubuntu the shell is Bash.

- Terminal (GNOME Terminal) shows text and passes keys; Bash reads your line and starts programs; the kernel does the work. Other shells: zsh, dash, ksh.
- History: Up/Down arrows step through earlier commands; history prints the numbered list; Ctrl+R searches backwards; ~/.bash_history stores the list when Bash exits.
- Tab completion: one Tab completes a single match (cd Doc + Tab = cd Documents/); Tab Tab lists choices; it also completes program names from PATH. If Tab does nothing, check spelling and capitals.
- Each terminal window/tab runs its own Bash with its own current directory.

2.3.1 echo

Purpose: Prints its arguments (after Bash has expanded variables) to standard output.

Syntax

```
echo [OPTION]... [STRING]...
```

Options / Flags

Option / Flag	Meaning
-n	Do not output the trailing newline
-e	Enable backslash escapes (\n, \t)
$VAR	Bash replaces it with the variable's value first

Example

```
$ echo $SHELL
$ echo $HOME $USER
```

- echo $SHELL prints the path of your login shell (a variable, see 2.7).
- echo $USER $HOME shows the system variables.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ echo $SHELL
/bin/bash
nasteha@ubuntu:~$ echo $HOME $USER
/home/nasteha nasteha
```

Screenshot: echo in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.3.2 history

Purpose: Prints the numbered list of commands you have run.

Syntax

```
history [N]
```

Options / Flags

Option / Flag	Meaning
N	Show only the last N entries
-c	Clear the history list
-w	Write the current history to ~/.bash_history
Up / Down	Step through earlier commands
Ctrl + R	Search backwards by typing part of a command

Example

```
$ pwd
$ ls
$ cd /etc
$ cd ~
$ history
```

- The saved list is ~/.bash_history, written when Bash exits (a second window may not see commands yet).
- Ctrl+R shows (reverse-i-search) and finds 'cd /etc'.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ pwd
/home/nasteha
nasteha@ubuntu:~$ ls
nasteha@ubuntu:~$ cd /etc
nasteha@ubuntu:/etc$ cd ~
nasteha@ubuntu:~$ history
    1  pwd
    2  ls
    3  cd /etc
    4  cd ~
    5  history
(reverse-i-search)`etc': cd /etc
```

Screenshot: history in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: history is an interactive-shell feature; the transcript follows Figure 2.7.

2.3.3 Tab completion

Purpose: Pressing Tab lets Bash finish a file or command name.

Syntax

```
<partial name> Tab   |   <partial name> Tab Tab
```

Options / Flags

Option / Flag	Meaning
Tab	Complete a single match
Tab Tab	List all matches
Case	Names are case-sensitive; wrong case = no completion

Example

```
$ cd Doc[Tab]
$ cd Documents/
$ cd D[Tab][Tab]
$ mkd[Tab]
$ mkdir
```

- cd Doc + Tab completes to cd Documents/; cd D + Tab Tab lists Desktop/ Documents/ Downloads/; mkd + Tab completes mkdir.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ cd Doc[Tab]
nasteha@ubuntu:~$ cd Documents/
nasteha@ubuntu:~$ cd D[Tab][Tab]
Desktop/  Documents/  Downloads/
nasteha@ubuntu:~$ mkd[Tab]
nasteha@ubuntu:~$ mkdir
```

Screenshot: Tab completion in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative: Tab is an interactive keystroke feature and cannot be captured by a script.

2.4 Command Syntax

Pattern: command, then options, then arguments. Only the command is required. Spaces separate the parts.

- Command = the program (ls, cp, mkdir). Options start with a dash and change behaviour (-l). Arguments are what it works on (/home).
- Short options: one dash, one letter, combinable (-la). Long options: two dashes, a whole word, not combinable (--all). -all is read as -a -l -l.
- Options usually come first; some commands care about order.

Short	Long	Meaning
-a	--all	include hidden names
-l	--format=long	one line per file, with details
-h	--human-readable	sizes as K, M, G
-r	--recursive	go into sub-directories (cp)
-i	--interactive	ask before overwriting or deleting

2.4.1 ls -l /home

Purpose: Anatomy example: command ls, option -l, argument /home. Also shows short and long option forms.

Syntax

```
ls [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-a	Short form of --all
--all	Include hidden names
-l	Long listing (= --format=long)
-la	Combined short options
--human-readable	Sizes as K, M, G
-all	WRONG: read as -a -l -l

Example

```
$ ls -l /home
$ ls -a
$ ls --all
$ ls -la
$ ls --all --format=long
$ ls -l --human-readable
```

- ls -a / ls -l / ls -la are short forms; ls --all and ls --human-readable are long forms.
- Label cp -i notes.txt backup: command cp, option -i, arguments notes.txt and backup.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls -l /home
total 12
drwxr-xr-x 8 root    root    4096 Nov 11 10:22 claude
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 nasteha
drwxr-x--- 2 ubuntu  ubuntu  4096 Oct 24 14:12 ubuntu
nasteha@ubuntu:~$ ls -a
. .. .bash_logout .bashrc .hidden .profile visible
nasteha@ubuntu:~$ ls --all
. .. .bash_logout .bashrc .hidden .profile visible
nasteha@ubuntu:~$ ls -la
total 20
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 .
drwxr-xr-x 5 root    root    4096 Nov 11 10:22 ..
-rw-r--r-- 1 nasteha nasteha  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 nasteha nasteha 3771 Mar 31  2024 .bashrc
-rw-r--r-- 1 nasteha nasteha    0 Nov 11 18:43 .hidden
-rw-r--r-- 1 nasteha nasteha  807 Mar 31  2024 .profile
-rw-r--r-- 1 nasteha nasteha    0 Nov 11 18:43 visible
nasteha@ubuntu:~$ ls --all --format=long
total 20
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 .
drwxr-xr-x 5 root    root    4096 Nov 11 10:22 ..
-rw-r--r-- 1 nasteha nasteha  220 Mar 31  2024 .bash_logout
```

```
-rw-r--r-- 1 nasteha nasteha 3771 Mar 31  2024 .bashrc
-rw-r--r-- 1 nasteha nasteha    0 Nov 11 18:43 .hidden
-rw-r--r-- 1 nasteha nasteha  807 Mar 31  2024 .profile
-rw-r--r-- 1 nasteha nasteha    0 Nov 11 18:43 visible
nasteha@ubuntu:~$ ls -l --human-readable
total 0
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43 visible
```

Screenshot: ls -l /home in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.5 Getting Help: man, --help, and info

Ubuntu carries its own documentation: manual pages, the short --help summary, and longer info documents.

- man opens a viewer; move with Space (next page), b (back), /word (search), n (next match), q (quit). The number in LS(1) is the manual section: 1 user commands, 5 file formats, 8 admin commands (man 1 passwd vs man 5 passwd).
- Read NAME and SYNOPSIS first; [ ] means optional, ... means repeatable. 'The terminal froze' = you are inside the man viewer; press q.
- cd is part of Bash itself, so 'man cd' says 'No manual entry for cd': use help cd.

2.5.1 man

Purpose: Opens the full manual page of a command in a viewer.

Syntax

```
man [SECTION] COMMAND
```

Options / Flags

Option / Flag	Meaning
q	Quit the viewer
Space	Next page
b	Back a page
/word	Search for word
n	Next match
SECTION	1 user commands, 5 file formats, 8 admin commands

Example

```
$ man ls
$ man cd
```

- man ls opens the ls page; man 1 passwd shows the command and man 5 passwd the file format; man rmdir (lab step 14) shows -p, --parents; man grep is the pre-reading for Chapter 3.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ man ls
LS(1)            User Commands            LS(1)
```

```
NAME
      ls - list directory contents

SYNOPSIS
      ls [OPTION]... [FILE]...

DESCRIPTION
      -a, --all
            do not ignore entries starting with .

 Manual page ls(1) line 1 (press h for help or q to quit)
nasteha@ubuntu:~$ man cd
No manual entry for cd
```

Screenshot: man in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: the sandbox used to build this document has no manual pages installed and man needs an interactive viewer.

2.5.2 --help

Purpose: Prints a short usage summary straight to the screen (no viewer, nothing to quit).

Syntax

```
COMMAND --help
```

Options / Flags

Option / Flag	Meaning
--help	Quick summary of options
-p, --parents	(mkdir) no error if existing, make parent directories as needed

Example

```
$ mkdir --help | head -n 8
```

- mkdir --help shows the usage line and the -p option.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ mkdir --help | head -n 8
Usage: mkdir [OPTION]... DIRECTORY...
Create the DIRECTORY(ies), if they do not already exist.

Mandatory arguments to long options are mandatory for short options too.
  -m, --mode=MODE   set file mode (as in chmod), not a=rwx - umask
  -p, --parents     no error if existing, make parent directories as needed,
                      with their file modes unaffected by any -m option.
  -v, --verbose     print a message for each created directory
```

Screenshot: --help in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.5.3 info

Purpose: Shows longer, chapter-style GNU documentation with links.

Syntax

```
info [COMMAND]
```

Options / Flags

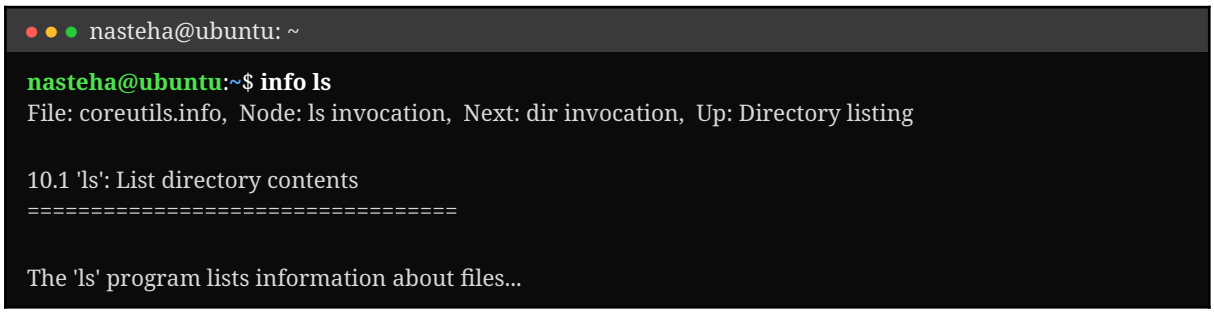
Option / Flag	Meaning
q	Quit info
Space	Next page
n / p	Next / previous node
Enter	Follow a link

Example

```
$ info ls
```

- info ls opens the long GNU manual for ls; use q to leave.

Terminal (screenshot of the command and its output)



Screenshot: info in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: the info program is not installed in the build sandbox.

2.5.4 help

Purpose: Shows help for Bash built-in commands such as cd (which have no man page).

Syntax

```
help [BUILTIN]
```

Options / Flags

Option / Flag	Meaning
help cd	Help for the cd built-in
-d	Short description
-s	Short usage synopsis

Example

```
$ help cd | head -n 6
$ help -s cd
```

- man cd fails because cd is built into Bash; help cd works.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ help cd | head -n 6
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
    HOME shell variable. If DIR is "-", it is converted to $OLDPWD.

nasteha@ubuntu:~$ help -s cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
```

Screenshot: help in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.6 Quoting, Escaping, and Wildcards

Bash changes some characters before a command sees them. Quotes and backslashes protect characters; wildcards ask Bash to fill in names.

- Double quotes keep spaces but still expand $. Single quotes keep everything exactly as typed. A backslash protects exactly the next character.
- Characters Bash treats specially: space $ * ? \ ' " & ; | < > # ~
- Wildcards belong to Bash, not to ls: * any characters or none; ? exactly one; [abc] one of the set; [0-9] one in the range. * never matches hidden names. If nothing matches, Bash passes the pattern unchanged.
- Look before you delete: run ls pattern first, then rm pattern. rm * .txt (with a space) = every file, then a file called .txt.

Wildcard	Matches	Example	Result
*	any characters, or none	ls *.txt	lab1.txt lab10.txt lab2.txt notes.txt
?	exactly one character	ls lab?.txt	lab1.txt lab2.txt
[abc]	one of the listed characters	ls lab[12].txt	lab1.txt lab2.txt
[0-9]	one character in the range	ls *[0-9].jpg	photo1.jpg

2.6.1 Quoting with " " and ' '

Purpose: Quotes group words and decide whether $ is expanded.

Syntax

```
echo "text"  |  echo 'text'
```

Options / Flags

Option / Flag	Meaning
"..."	Keep spaces, still expand $VARIABLES

Option / Flag	Meaning
'...'	Keep everything exactly as typed
(no quotes)	Words split at spaces; extra spaces collapse

Example

```
$ echo Hello     $USER
$ echo "Hello     $USER"
$ echo 'Hello     $USER'
$ touch new file
$ ls
$ touch "new file"
$ ls
```

- echo Hello $USER -> two words; double quotes keep the spaces; single quotes keep $USER literally.
- touch new file creates two files; touch "new file" creates one.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ echo Hello     $USER
Hello nasteha
nasteha@ubuntu:~/lab$ echo "Hello     $USER"
Hello     nasteha
nasteha@ubuntu:~/lab$ echo 'Hello     $USER'
Hello     $USER
nasteha@ubuntu:~/lab$ touch new file
nasteha@ubuntu:~/lab$ ls
file  new
nasteha@ubuntu:~/lab$ touch "new file"
nasteha@ubuntu:~/lab$ ls
 file   new  'new file'
```

Screenshot: Quoting with " " and ' ' in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.6.2 Escaping with a backslash

Purpose: A backslash protects the single character that follows it.

Syntax

```
COMMAND char\ with\ special
```

Options / Flags

Option / Flag	Meaning
\ (space)	cd lab\ report - the space is protected
\$	echo \$HOME prints $HOME literally
*	touch notes*.txt - the star is part of the name

Example

```
$ mkdir 'lab report'
$ cd lab\ report
```

```
$ pwd
$ cd ..
$ echo \$HOME
$ echo $HOME
```

- Press Tab on a name with a space and Bash adds the backslash for you.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ mkdir 'lab report'
nasteha@ubuntu:~$ cd lab\ report
nasteha@ubuntu:~/lab report$ pwd
/home/nasteha/lab report
nasteha@ubuntu:~/lab report$ cd ..
nasteha@ubuntu:~$ echo \$HOME
$HOME
nasteha@ubuntu:~$ echo $HOME
/home/nasteha
```

Screenshot: Escaping with a backslash in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.6.3 Wildcards * ? [ ]

Purpose: Globbing: Bash expands the pattern into matching names before the command runs.

Syntax

```
ls PATTERN
```

Options / Flags

Option / Flag	Meaning
*****	Any characters, or none
?	Exactly one character
[abc]	One of the listed characters
[0-9]	One character in the range

Example

```
$ ls *.txt
$ ls lab?.txt
$ ls lab[12].txt
$ ls *[0-9].jpg
$ ls *.xyz
```

- ls never sees the *; it receives the four file names.
- Create the six files first, then run each pattern.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ ls *.txt
lab1.txt  lab10.txt  lab2.txt  notes.txt
nasteha@ubuntu:~/lab$ ls lab?.txt
lab1.txt  lab2.txt
```

```
nasteha@ubuntu:~/lab$ ls lab[12].txt
lab1.txt  lab2.txt
nasteha@ubuntu:~/lab$ ls *[0-9].jpg
photo1.jpg
nasteha@ubuntu:~/lab$ ls *.xyz
ls: cannot access '*.xyz': No such file or directory
```

*Screenshot: Wildcards * ? [ ] in the Ubuntu terminal -* **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7 Variables, PATH, and ~/.bashrc

Bash keeps small pieces of information in variables. PATH decides whether typing a command works at all.

- NAME=value sets a shell variable (no spaces around =); $NAME reads it. 'COURSE = linux' fails: Bash reads COURSE as a command. Names are case-sensitive; system variables are capitals (HOME, USER, PATH).
- export passes a variable to programs started from this shell (the environment). Values set at the prompt vanish when the terminal closes.
- PATH is a colon-separated list of directories searched left to right; the first match runs; the current directory is NOT searched (run a local program as ./program). Add safely with PATH="$HOME/bin:$PATH".
- ~/.bashrc: Bash reads /etc/bash.bashrc then ~/.bashrc when a terminal opens. It holds export lines, aliases (Ubuntu defines ll as ls -alF) and the prompt. Changes affect new terminals; reload with source ~/.bashrc.

2.7.1 Variables: NAME=value

Purpose: Creates a shell variable; $VAR reads it.

Syntax

```
NAME=value
echo $NAME
```

Options / Flags

Option / Flag	Meaning
NAME=value	No spaces around =
$NAME	The value of NAME
NAME = value	ERROR: command not found

Example

```
$ COURSE=linux
$ echo $COURSE
$ echo $HOME $USER
$ COURSE = linux
```

- COURSE=linux then echo $COURSE prints linux. With spaces around = Bash says command not found.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ COURSE=linux
nasteha@ubuntu:~$ echo $COURSE
linux
nasteha@ubuntu:~$ echo $HOME $USER
/home/nasteha nasteha
nasteha@ubuntu:~$ COURSE = linux
bash: COURSE: command not found
```

Screenshot: Variables: NAME=value in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7.2 export

Purpose: Marks a variable so programs started from this shell receive a copy (the environment).

Syntax

```
export [-n] NAME[=value]
```

Options / Flags

Option / Flag	Meaning
-p	List all exported variables
-n	Remove the export property
NAME=value	Set and export in one step

Example

```
$ COURSE=linux
$ export COURSE
$ bash -c 'echo child sees: $COURSE'
$ export -p | grep COURSE
```

- A new terminal window will not know COURSE: export only passes it to child programs.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ COURSE=linux
nasteha@ubuntu:~$ export COURSE
nasteha@ubuntu:~$ bash -c 'echo child sees: $COURSE'
child sees: linux
nasteha@ubuntu:~$ export -p | grep COURSE
declare -x COURSE="linux"
```

Screenshot: export in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7.3 echo $PATH

Purpose: Prints the list of directories Bash searches for programs.

Syntax

```
echo $PATH
```

Options / Flags

Option / Flag	Meaning
$PATH	Colon-separated directory list
./program	Run a program in the current directory
PATH="$HOME/bin:$PATH"	Add a directory (do not replace the list)

Example

```
$ echo $PATH
$ ls /usr/bin/ls
$ sl
```

- Bash tries each directory left to right and runs the first match. ls is found in /usr/bin.
- A name that is not found prints 'command not found' (Ubuntu also suggests how to install it).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
nasteha@ubuntu:~$ ls /usr/bin/ls
/usr/bin/ls
nasteha@ubuntu:~$ sl
bash: sl: command not found
```

Screenshot: echo $PATH in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7.4 ~/.bashrc

Purpose: Your personal Bash settings file (hidden). Listed with ls -a ~; changes apply to new terminals.

Syntax

```
ls -a ~
source ~/.bashrc
```

Options / Flags

Option / Flag	Meaning
ls -a ~	Shows hidden files: .bashrc .profile .bash_history ...
source ~/.bashrc	Reload settings into the current terminal
/etc/bash.bashrc	System-wide defaults read first

Example

```
$ ls -a ~
$ grep -n "alias ll" ~/.bashrc
```

- Find .bashrc in the hidden listing. The alias ll (ls -alF) is defined in Ubuntu's copy.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ ls -a ~
. .. .bash_logout .bashrc .profile
nasteha@ubuntu:~$ grep -n "alias ll" ~/.bashrc
91:alias ll='ls -alF'
```

Screenshot: ~/.bashrc in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7.5 source

Purpose: Runs a file in the current shell, so ~/.bashrc changes apply immediately.

Syntax

```
source FILE   (or: . FILE)
```

Options / Flags

Option / Flag	Meaning
source ~/.bashrc	Reload your Bash settings
. FILE	Synonym of source

Example

```
$ echo 'export MYTOOL=on' >> ~/.bashrc
$ echo $MYTOOL
$ source ~/.bashrc
$ echo $MYTOOL
```

- Changing ~/.bashrc does not change the current terminal unless you reload it.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ echo 'export MYTOOL=on' >> ~/.bashrc
nasteha@ubuntu:~$ echo $MYTOOL
nasteha@ubuntu:~$ source ~/.bashrc
nasteha@ubuntu:~$ echo $MYTOOL
on
```

Screenshot: source in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.7.6 alias (ll)

Purpose: A short name for a long command; Ubuntu's ~/.bashrc defines ll as ls -alF.

Syntax

```
alias NAME='command'
```

Options / Flags

Option / Flag	Meaning
alias	List aliases
alias ll='ls -alF'	Define the Ubuntu ll alias
unalias NAME	Remove an alias

Example

```
$ alias ll='ls -alF'
$ alias ll
$ mkdir d
$ ll
```

- The ll alias is set in the interactive Bash of Ubuntu; here it is defined by hand and used.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ alias ll='ls -alF'
nasteha@ubuntu:~$ alias ll
alias ll='ls -alF'
nasteha@ubuntu:~$ mkdir d
nasteha@ubuntu:~$ ll
total 24
drwxr-xr-x 3 nasteha nasteha 4096 Nov 11 18:43 ./
drwxr-xr-x 5 root    root    4096 Nov 11 10:22 ../
-rw-r--r-- 1 nasteha nasteha  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 nasteha nasteha 3771 Mar 31  2024 .bashrc
-rw-r--r-- 1 nasteha nasteha  807 Mar 31  2024 .profile
drwxr-xr-x 2 nasteha nasteha 4096 Nov 11 18:43 d/
```

Screenshot: alias (ll) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.8 Paths and Filenames

A path is an address in the tree: absolute from /, or relative from where you stand.

- Absolute: cd /home/nasteha/Downloads (same anywhere). Relative: cd ../Downloads (up one level, then into Downloads). 'Documents/report.txt' and '/Documents/report.txt' differ.
- . = this directory; .. = parent (the .. of / is itself); ~ = your home, expanded by Bash.
- Case matters (notes.txt, Notes.txt, NOTES.txt are three files). A leading dot hides a name (ls -a shows it); hidden does not mean protected. Extensions are only labels. Never allowed: / and the empty name; max 255 bytes.
- Use letters, digits and - _ . only; do not start a name with - (read as an option).

Sensible	Awkward
lab-report_v2.txt	Lab Report (final).txt
week3	week 3
photo_2026-03-01.jpg	my*photo?.jpg
backup.old	-backup

2.8.1 cd with absolute and relative paths

Purpose: Changes the current directory using absolute or relative paths and the shorthand names . .. ~.

Syntax

```
cd [DIRECTORY]
```

Options / Flags

Option / Flag	Meaning
/abs/path	Absolute path from /
../Downloads	Relative: up one, then into Downloads
..	Parent directory
.	Current directory
~	Home directory

Example

```
$ cd ..
$ pwd
$ cd ../..
$ pwd
$ cd ~/Documents
$ cd .
$ pwd
$ cd ../Downloads
$ pwd
$ cd /home/nasteha/Downloads
$ pwd
```

- From Documents: cd .. goes to home; cd ../.. goes to /; cd ~/Documents returns; cd . stays.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/Documents
nasteha@ubuntu:~/Documents$ cd ..
nasteha@ubuntu:~$ pwd
/home/nasteha
nasteha@ubuntu:~$ cd ../..
nasteha@ubuntu:/$ pwd
/
nasteha@ubuntu:/$ cd ~/Documents
nasteha@ubuntu:~/Documents$ cd .
nasteha@ubuntu:~/Documents$ pwd
/home/nasteha/Documents
nasteha@ubuntu:~/Documents$ cd ../Downloads
nasteha@ubuntu:~/Downloads$ pwd
/home/nasteha/Downloads
nasteha@ubuntu:~/Downloads$ cd /home/nasteha/Downloads
nasteha@ubuntu:~/Downloads$ pwd
/home/nasteha/Downloads
```

Screenshot: cd with absolute and relative paths in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.8.2 Case sensitivity and hidden names

Purpose: Linux names are case-sensitive; a leading dot hides a name.

Syntax

```
ls -a
```

Options / Flags

Option / Flag	Meaning
Notes.txt vs notes.txt	Two different files
.name	Hidden; shown by ls -a
cd documents	Fails if the folder is Documents

Example

```
$ touch Notes.txt notes.txt .hidden
$ ls
$ ls -a
$ mkdir Documents
$ cd documents
```

- Create Notes.txt and notes.txt side by side; cd documents fails when the folder is Documents.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ touch Notes.txt notes.txt .hidden
nasteha@ubuntu:~$ ls
Notes.txt  notes.txt
nasteha@ubuntu:~$ ls -a
.  ..  .bash_logout  .bashrc  .hidden  .profile  Notes.txt  notes.txt
nasteha@ubuntu:~$ mkdir Documents
nasteha@ubuntu:~$ cd documents
bash: cd: documents: No such file or directory
```

Screenshot: Case sensitivity and hidden names in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.9 Navigating with pwd, cd, and ls

pwd tells you where you are, cd moves you, ls shows what is there. cd only moves; pwd and ls only report.

- cd path moves; cd alone goes home; cd - goes back to the previous directory and prints it; .. goes up. Lost? Type cd, then pwd.
- ls -a adds hidden names; -l gives one line per entry (first character d = directory, - = regular file; the other nine letters are permissions, Chapter 5); -h prints sizes as K, M, G; options combine (-lah).
- A directory's 4.0K is the directory entry itself, not its contents.

2.9.1 cd (pwd and cd together)

Purpose: Changes the current directory.

Syntax

```
cd [DIRECTORY]
```

Options / Flags

Option / Flag	Meaning
cd /var/log	Absolute path
cd Documents	Relative path

Option / Flag	Meaning
cd ..	Up one level
cd or cd ~	Home
cd -	Previous directory (also prints it)

Example

```
$ pwd
$ cd /var/log
$ cd ..
$ cd -
$ cd
$ pwd
```

- Follow the prompt: it shows the new directory after every cd. cd - goes back.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ pwd
/home/nasteha
nasteha@ubuntu:~$ cd /var/log
nasteha@ubuntu:/var/log$ cd ..
nasteha@ubuntu:/var$ cd -
/var/log
nasteha@ubuntu:/var/log$ cd
nasteha@ubuntu:~$ pwd
/home/nasteha
```

Screenshot: cd (pwd and cd together) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.9.2 ls with -a, -l and -h

Purpose: Continues ls from Chapter 1: hidden names, long listing, readable sizes.

Syntax

```
ls [-a] [-l] [-h] [PATH]
```

Options / Flags

Option / Flag	Meaning
ls	Names only
ls -a	All names including . .. and hidden
ls -l	Long listing: type+permissions, links, owner, group, size, modified, name
ls -lh	Long listing with K, M, G sizes
ls -lah	Everything with readable sizes
ls -l /etc	List another place

Example

```
$ ls
$ ls -a
```

```
$ ls -l
$ ls -lh
$ ls -lah
```

- Compare ls and ls -a in your home: which new names appear?

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls
Documents  notes.txt  slides.pdf
nasteha@ubuntu:~$ ls -a
.  ..  .bash_logout  .bashrc  .profile  Documents  notes.txt  slides.pdf
nasteha@ubuntu:~$ ls -l
total 2352
drwxr-xr-x 2 nasteha nasteha    4096 Nov 11 18:43 Documents
-rw-r--r-- 1 nasteha nasteha       6 Nov 11 18:43 notes.txt
-rw-r--r-- 1 nasteha nasteha 2400000 Nov 11 18:43 slides.pdf
nasteha@ubuntu:~$ ls -lh
total 2.3M
drwxr-xr-x 2 nasteha nasteha 4.0K Nov 11 18:43 Documents
-rw-r--r-- 1 nasteha nasteha    6 Nov 11 18:43 notes.txt
-rw-r--r-- 1 nasteha nasteha 2.3M Nov 11 18:43 slides.pdf
nasteha@ubuntu:~$ ls -lah
total 2.4M
drwxr-xr-x 3 nasteha nasteha 4.0K Nov 11 18:43 .
drwxr-xr-x 5 root    root    4.0K Nov 11 10:22 ..
-rw-r--r-- 1 nasteha nasteha  220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 nasteha nasteha 3.7K Nov 11 18:43 .bashrc
-rw-r--r-- 1 nasteha nasteha  807 Mar 31  2024 .profile
drwxr-xr-x 2 nasteha nasteha 4.0K Nov 11 18:43 Documents
-rw-r--r-- 1 nasteha nasteha    6 Nov 11 18:43 notes.txt
-rw-r--r-- 1 nasteha nasteha 2.3M Nov 11 18:43 slides.pdf
```

Screenshot: ls with -a, -l and -h in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10 Creating and Managing Files

Now we change things. rm has no recycle bin: once removed from the terminal, a file is gone. Practise only inside ~/lab.

- touch makes an empty file or refreshes the time of an existing one; mkdir -p builds a whole path; rmdir removes empty directories only.
- cp copies (-r for directories); mv moves or renames; both overwrite silently, -i asks first.
- rm: no Trash, no undo, no confirmation by default. rm -i asks; rm -r removes a directory with contents; rm -rf is forced with no questions: avoid. Before pressing Enter: pwd, then ls with the same pattern, then rm.
- sudo runs one command as administrator (Chapter 5). 'sudo rm -rf /' is not a joke: never type it and never paste commands you cannot read.

2.10.1 touch

Purpose: Creates an empty file, or updates the timestamp of an existing one.

Syntax

```
touch [OPTION]... FILE...
```

Options / Flags

Option / Flag	Meaning
-a	Change only the access time
-m	Change only the modification time
-c	Do not create missing files
-t STAMP	Use a given time
FILE...	One or more names

Example

```
$ touch notes.txt todo.md
$ ls -l
```

- touch notes.txt todo.md creates two empty files.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ touch notes.txt todo.md
nasteha@ubuntu:~/lab$ ls -l
total 0
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43 notes.txt
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43 todo.md
```

Screenshot: touch in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.2 mkdir

Purpose: Creates directories.

Syntax

```
mkdir [-p] DIRECTORY...
```

Options / Flags

Option / Flag	Meaning
-p, --parents	No error if existing; create missing parents
-v	Print a message for each created directory
-m MODE	Set permissions

Example

```
$ mkdir week3
$ mkdir -p projects/linux
$ ls
```

```
$ mkdir x/y/z
```

- mkdir -p projects/linux builds both levels in one go; without -p, mkdir a/b/c fails if a does not exist.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ mkdir week3
nasteha@ubuntu:~/lab$ mkdir -p projects/linux
nasteha@ubuntu:~/lab$ ls
projects  week3
nasteha@ubuntu:~/lab$ mkdir x/y/z
mkdir: cannot create directory ‘x/y/z’: No such file or directory
```

Screenshot: mkdir in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.3 rmdir

Purpose: Removes directories, but only if they are empty (its safety feature).

Syntax

```
rmdir [-p] DIRECTORY...
```

Options / Flags

Option / Flag	Meaning
-p, --parents	Also remove empty parent directories
-v	Print a message for each removal
--ignore-fail-on-non-empty	Do not complain about non-empty directories

Example

```
$ rmdir projects
$ rmdir week3
$ ls
$ rmdir -p a/b/c
$ ls
```

- rmdir refuses 'projects' because it holds linux; it works on week3.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ rmdir projects
rmdir: failed to remove 'projects': Directory not empty
nasteha@ubuntu:~/lab$ rmdir week3
nasteha@ubuntu:~/lab$ ls
a  projects
nasteha@ubuntu:~/lab$ rmdir -p a/b/c
nasteha@ubuntu:~/lab$ ls
projects
```

Screenshot: rmdir in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.4 cp

Purpose: Copies files or directories; the original stays.

Syntax

```
cp [OPTION]... SOURCE DEST
```

Options / Flags

Option / Flag	Meaning
-r, -R	Copy directories recursively
-i, --interactive	Ask before overwriting
-v	Show what is copied
-p	Preserve mode, ownership, timestamps
-n	Do not overwrite an existing file

Example

```
$ cp notes.txt notes.bak
$ ls
$ cp notes.txt backup/
$ cp -r projects projects-copy
$ ls
$ cp -i notes.txt notes.bak
```

- cp notes.txt notes.bak leaves two files; cp notes.txt backup/ puts a copy in backup; cp -r projects projects-copy copies a directory.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ cp notes.txt notes.bak
nasteha@ubuntu:~/lab$ ls
backup  notes.bak  notes.txt  projects
nasteha@ubuntu:~/lab$ cp notes.txt backup/
nasteha@ubuntu:~/lab$ cp -r projects projects-copy
nasteha@ubuntu:~/lab$ ls
backup  notes.bak  notes.txt  projects  projects-copy
nasteha@ubuntu:~/lab$ cp -i notes.txt notes.bak
cp: overwrite 'notes.bak'? n
```

Screenshot: cp in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.5 mv

Purpose: Moves or renames files and directories (renaming is moving within the same directory).

Syntax

```
mv [OPTION]... SOURCE DEST
```

Options / Flags

Option / Flag	Meaning
-i, --interactive	Ask before overwriting
-n	Do not overwrite

Option / Flag	Meaning
-v	Show what is moved
-u	Move only when the source is newer
mv OLD NEW	Rename
mv FILE DIR/	Move into a directory

Example

```
$ mv notes.txt old.txt
$ ls
$ mv draft.txt final.txt
$ mv final.txt ~/Documents/
$ ls ~/Documents
$ touch other.txt
$ mv -i other.txt old.txt
```

- mv notes.txt old.txt renames; mv final.txt ~/Documents/ moves; mv draft.txt ~/Documents/final.txt renames and moves in one command.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ mv notes.txt old.txt
nasteha@ubuntu:~/lab$ ls
draft.txt  old.txt
nasteha@ubuntu:~/lab$ mv draft.txt final.txt
nasteha@ubuntu:~/lab$ mv final.txt ~/Documents/
nasteha@ubuntu:~/lab$ ls ~/Documents
final.txt
nasteha@ubuntu:~/lab$ touch other.txt
nasteha@ubuntu:~/lab$ mv -i other.txt old.txt
mv: overwrite 'old.txt'? n
```

Screenshot: mv in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.6 rm

Purpose: Removes files (and directories with -r). There is no Trash and no undo.

Syntax

```
rm [OPTION]... FILE...
```

Options / Flags

Option / Flag	Meaning
-i	Ask before each removal (answer y or n)
-r, -R	Remove directories and their contents
-f	Force: no questions, no errors (avoid with -r)
-v	Show what is removed
-d	Remove empty directories

Example

```
$ pwd
$ ls *.bak
$ rm -i notes.bak
$ ls
$ rm -i notes.bak
$ ls
$ rm -r dir1
$ ls
```

- Safe order: pwd, ls with the same pattern, then rm. The prompts are answered n, then y (answers typed at the prompt).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/lab
nasteha@ubuntu:~/lab$ pwd
/home/nasteha/lab
nasteha@ubuntu:~/lab$ ls *.bak
notes.bak
nasteha@ubuntu:~/lab$ rm -i notes.bak
rm: remove regular empty file 'notes.bak'? n
nasteha@ubuntu:~/lab$ ls
dir1  notes.bak
nasteha@ubuntu:~/lab$ rm -i notes.bak
rm: remove regular empty file 'notes.bak'? y
nasteha@ubuntu:~/lab$ ls
dir1
nasteha@ubuntu:~/lab$ rm -r dir1
nasteha@ubuntu:~/lab$ ls
```

Screenshot: rm in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.10.7 sudo

Purpose: Runs one command with administrator (root) rights; studied in Chapter 5. Mentioned with apt install, kill, renice, systemctl and a warning about rm -rf /.

Syntax

```
sudo COMMAND
```

Options / Flags

Option / Flag	Meaning
sudo apt install sl	Install the sl package (Ubuntu hint when a command is missing)
sudo kill -HUP PID	Signal a process you do not own
sudo rm -rf /	NEVER type this

Example

```
$ sl
$ sudo apt install sl
```

- The first sudo use asks for your password; the sandbox used for this document has no sudo, so a representative transcript is shown.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ sl
Command 'sl' not found, but can be installed with:
sudo apt install sl
nasteha@ubuntu:~$ sudo apt install sl
[sudo] password for nasteha:
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
  sl
Setting up sl (5.02-1) ...
```

Screenshot: sudo in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (needs administrator rights and network access). WARNING: never run sudo rm -rf /.

2.11 Lab: Navigating and Managing Files

Work in ~/ch2lab and record what you see in the evidence column (Figures 2.27 and 2.28).

- Step 5 quotes keep 'week 3.txt' as one name; forgetting them gives 'week' and '3.txt'. Step 11: * matched three names including 'week 3.txt'. Step 14 answer: man rmdir > -p, --parents (search with /parent).
- Keep ~/ch2lab: it is used again in Chapter 3.

2.11.1 Lab Part A (steps 1-7)

Purpose: Build and navigate: cd, mkdir -p, mkdir, touch with quotes, ls -l, ls -a.

Syntax

> See table

Options / Flags

Option / Flag	Meaning
1 cd	Home
2 mkdir -p ~/ch2lab/notes	Create ch2lab and notes
3 mkdir ~/ch2lab/backup	Second directory
4 cd ~/ch2lab	Enter it; check with pwd
5 touch notes/day1.txt notes/day2.txt "notes/week 3.txt"	Three files
6 ls -l notes	Three lines starting with -
7 ls -a ~	Hidden names; find .bashrc

Example

```
$ cd
$ mkdir -p ~/ch2lab/notes
$ mkdir ~/ch2lab/backup
$ cd ~/ch2lab
```

```
$ pwd
$ touch notes/day1.txt notes/day2.txt "notes/week 3.txt"
$ ls -l notes
$ ls -a ~
```

- Type each line yourself; the third column of the evidence sheet is the output below.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ cd
nasteha@ubuntu:~$ mkdir -p ~/ch2lab/notes
nasteha@ubuntu:~$ mkdir ~/ch2lab/backup
nasteha@ubuntu:~$ cd ~/ch2lab
nasteha@ubuntu:~/ch2lab$ pwd
/home/nasteha/ch2lab
nasteha@ubuntu:~/ch2lab$ touch notes/day1.txt notes/day2.txt "notes/week 3.txt"
nasteha@ubuntu:~/ch2lab$ ls -l notes
total 0
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43  day1.txt
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43  day2.txt
-rw-r--r-- 1 nasteha nasteha 0 Nov 11 18:43 'week 3.txt'
nasteha@ubuntu:~/ch2lab$ ls -a ~
. .. .bash_logout  .bashrc  .profile  ch2lab
```

Screenshot: Lab Part A (steps 1-7) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

2.11.2 Lab Part B (steps 8-14)

Purpose: Copy, move, remove and look things up: cp, mv, cp -r, rm -i, rmdir, echo $PATH, man.

Syntax

```
See table
```

Options / Flags

Option / Flag	Meaning
8 cp notes/day1.txt backup/	Copy
9 mv notes/day2.txt notes/monday.txt	Rename
10 cp -r notes notes-copy	Copy directory
11 rm -i notes-copy/*	Asks three times
12 rmdir notes-copy	Works when empty
13 echo $PATH	Find /usr/bin
14 man rmdir	Find -p, --parents

Example

```
$ cp notes/day1.txt backup/
$ mv notes/day2.txt notes/monday.txt
$ ls notes
$ cp -r notes notes-copy
$ rm -i notes-copy/*
$ rmdir notes-copy
```

```
$ echo $PATH
$ ls /usr/bin/mkdir
```

- rm -i is answered y three times; man rmdir is interactive so it is shown as representative (last lines).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/ch2lab
nasteha@ubuntu:~/ch2lab$ cp notes/day1.txt backup/
nasteha@ubuntu:~/ch2lab$ mv notes/day2.txt notes/monday.txt
nasteha@ubuntu:~/ch2lab$ ls notes
 day1.txt   monday.txt  'week 3.txt'
nasteha@ubuntu:~/ch2lab$ cp -r notes notes-copy
nasteha@ubuntu:~/ch2lab$ rm -i notes-copy/*
rm: remove regular empty file 'notes-copy/day1.txt'? y
rm: remove regular empty file 'notes-copy/monday.txt'? y
rm: remove regular empty file 'notes-copy/week 3.txt'? y
nasteha@ubuntu:~/ch2lab$ rmdir notes-copy
nasteha@ubuntu:~/ch2lab$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
nasteha@ubuntu:~/ch2lab$ ls /usr/bin/mkdir
/usr/bin/mkdir
```

Screenshot: Lab Part B (steps 8-14) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

Note: Step 14 (man rmdir): the option that also removes empty parents is -p, --parents.

Chapter 3: Text Processing, Searching, Archiving, and Editors

Chapter 3 overview - commands in this chapter

Command / topic	Section	No.	Output
mkdir -p and cd (lab folder)	Preparing a Safe Practice Workspace	3.0.1	Real
printf	Preparing a Safe Practice Workspace	3.0.2	Real
cat (continued: -n, several files)	Viewing and Searching Files	3.1.1	Real
seq	Viewing and Searching Files	3.1.2	Real
less	Viewing and Searching Files	3.1.3	Representative
more	Viewing and Searching Files	3.1.4	Representative
head	Viewing and Searching Files	3.1.5	Real
tail	Viewing and Searching Files	3.1.6	Real
tail -f (monitoring changes)	Viewing and Searching Files	3.1.7	Representative
file	Viewing and Searching Files	3.1.8	Real
stat	Viewing and Searching Files	3.1.9	Real
ls -l (basic metadata)	Viewing and Searching Files	3.1.10	Real
which	Viewing and Searching Files	3.1.11	Real
type	Viewing and Searching Files	3.1.12	Real
command -v	Viewing and Searching Files	3.1.13	Real
whereis	Viewing and Searching Files	3.1.14	Real
find	Viewing and Searching Files	3.1.15	Real
locate	Viewing and Searching Files	3.1.16	Representative
updatedb	Viewing and Searching Files	3.1.17	Representative
grep	Viewing and Searching Files	3.1.18	Real
grep with regular expressions	Viewing and Searching Files	3.1.19	Real

Command / topic	Section	No.	Output
Standard streams: stdout vs stderr	Redirection, Pipes, and Text Processing	3.2.1	Real
Output redirection > and >>	Redirection, Pipes, and Text Processing	3.2.2	Real
Error redirection 2>	Redirection, Pipes, and Text Processing	3.2.3	Real
Input redirection <	Redirection, Pipes, and Text Processing	3.2.4	Real
Pipes |	Redirection, Pipes, and Text Processing	3.2.5	Real
Command chaining ; && ||	Redirection, Pipes, and Text Processing	3.2.6	Real
test	Redirection, Pipes, and Text Processing	3.2.7	Real
sort	Redirection, Pipes, and Text Processing	3.2.8	Real
uniq	Redirection, Pipes, and Text Processing	3.2.9	Real
cut	Redirection, Pipes, and Text Processing	3.2.10	Real
paste	Redirection, Pipes, and Text Processing	3.2.11	Real
wc	Redirection, Pipes, and Text Processing	3.2.12	Real
tr	Redirection, Pipes, and Text Processing	3.2.13	Real
Pipeline: login shells	Redirection, Pipes, and Text Processing	3.2.14	Real
Pipeline: log messages	Redirection, Pipes, and Text Processing	3.2.15	Real
Pipeline: common words	Redirection, Pipes, and Text Processing	3.2.16	Real
tar	Archives and Compression	3.3.1	Real
gzip	Archives and Compression	3.3.2	Real
gunzip	Archives and Compression	3.3.3	Real
bzip2	Archives and Compression	3.3.4	Real
bzcat	Archives and Compression	3.3.5	Real
bunzip2	Archives and Compression	3.3.6	Real
xz	Archives and Compression	3.3.7	Real
xzcat	Archives and Compression	3.3.8	Real
unxz	Archives and Compression	3.3.9	Real
zip	Archives and Compression	3.3.10	Real
unzip	Archives and Compression	3.3.11	Real
nano	Terminal Text Editors: Nano and Vim	3.4.1	Representative

Command / topic	Section	No.	Output
vim	Terminal Text Editors: Nano and Vim	3.4.2	Representative
vi	Terminal Text Editors: Nano and Vim	3.4.3	Representative
vimtutor	Terminal Text Editors: Nano and Vim	3.4.4	Representative
ln	Hard Links and Symbolic Links	3.5.1	Real
ln -s	Hard Links and Symbolic Links	3.5.2	Real
ls -li	Hard Links and Symbolic Links	3.5.3	Real

3.0 Preparing a Safe Practice Workspace

Set up a repeatable lab directory before running the examples. No sudo is needed; all commands run as your normal user. > overwrites, >> appends (Section 3.2).

- Files created: names.txt (Ali, Hodan, Yusuf), cities.txt (Mogadishu appears three times), app.log (exactly two ERROR lines), notes.txt, words.txt, scores.txt, users.txt, project/README.md, project/config.txt, project/src/app.py.
- Everything stays inside ~/linux-ch3-lab so no personal documents or system configuration are touched.

3.0.1 mkdir -p and cd (lab folder)

Purpose: Creates the practice directory (and parents) and enters it.

Syntax

```
mkdir -p DIRECTORY
cd DIRECTORY
```

Options / Flags

Option / Flag	Meaning
-p	Create parents; no error if it exists
~	Your home directory

Example

```
$ mkdir -p ~/linux-ch3-lab
$ cd ~/linux-ch3-lab
$ pwd
```

- Slide 3: mkdir -p ~/linux-ch3-lab, then cd ~/linux-ch3-lab.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ mkdir -p ~/linux-ch3-lab
nasteha@ubuntu:~$ cd ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ pwd
```

```
/home/nasteha/linux-ch3-lab
```

Screenshot: mkdir -p and cd (lab folder) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.0.2 printf

Purpose: Prints formatted text; with > or >> it writes sample data files without an editor.

Syntax

```
printf FORMAT [ARGUMENT]...
```

Options / Flags

Option / Flag	Meaning
\n	Newline
\t	Tab
%s	Insert a string argument
> FILE	Overwrite FILE with the output
>> FILE	Append to FILE

Example

```
$ printf 'Ali\nHodan\nYusuf\n' > names.txt
$ printf 'Mogadishu\nHargeisa\n' > cities.txt
$ printf 'Mogadishu\nGarowe\nMogadishu\n' >> cities.txt
$ printf 'INFO start\nERROR disk full\n' > app.log
$ printf 'WARN retry\nERROR login failed\n' >> app.log
$ mkdir -p project
$ printf 'Linux project\n' > project/README.md
$ printf 'Linux tools help. Linux is useful.\n' > notes.txt
$ printf 'gray\ngrey\n' > words.txt
$ printf '42\n100\n7\n' > scores.txt
$ printf 'ali:student:bash\nhodan:student:sh\nyusuf:admin:bash\n' > users.txt
$ mkdir -p project/src
$ printf 'mode=practice\n' > project/config.txt
$ printf '# TODO: test\n' > project/src/app.py
$ ls -1
```

- Slides 3-5 build every sample file; ls -1 lists one name per line (-1 = one per line).

Terminal (screenshot of the command and its output)

```
● ● ● nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Ali\nHodan\nYusuf\n' > names.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Mogadishu\nHargeisa\n' > cities.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Mogadishu\nGarowe\nMogadishu\n' >> cities.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'INFO start\nERROR disk full\n' > app.log
nasteha@ubuntu:~/linux-ch3-lab$ printf 'WARN retry\nERROR login failed\n' >> app.log
nasteha@ubuntu:~/linux-ch3-lab$ mkdir -p project
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Linux project\n' > project/README.md
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Linux tools help. Linux is useful.\n' > notes.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'gray\ngrey\n' > words.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf '42\n100\n7\n' > scores.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'ali:student:bash\nhodan:student:sh\nyusuf:admin:bash\n' >
```

```
users.txt
nasteha@ubuntu:~/linux-ch3-lab$ mkdir -p project/src
nasteha@ubuntu:~/linux-ch3-lab$ printf 'mode=practice\n' > project/config.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf '# TODO: test\n' > project/src/app.py
nasteha@ubuntu:~/linux-ch3-lab$ ls -1
app.log
cities.txt
names.txt
notes.txt
project
scores.txt
users.txt
words.txt
```

Screenshot: printf in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1 Viewing and Searching Files

Read file content, describe metadata, locate paths and search inside text.

- find, locate and which search for paths/commands; grep searches inside file content.
- Executable command vs shell built-in: ls, cat, python3 are separate programs (/usr/bin/ls, check with which ls); cd, export, alias are built into Bash (check with type cd). 'which cd' may print nothing.
- find size operators: + greater than, - less than, none = exact; units c bytes, k KiB, M MiB, G GiB. Tests on one find command are AND conditions. Quote patterns ('*.log') so the shell does not expand them.
- find scans live (sees new files at once); locate uses a prebuilt index (fast but may be stale; updatedb refreshes it; plocate may need installing).

Metadata	Meaning	Example
File name	Name of the file	names.txt
File size	Storage used	16 bytes
File type	File or directory	Regular file
Owner	User who owns the file	current user
Permissions	Who can access	rw-r--r--
Access time	Last access	varies
Modification time	Last content change	varies
Inode	Filesystem identification number	varies

3.1.1 cat (continued: -n, several files)

Purpose: Displays file contents (concatenate); several files print in sequence. Not a pager.

Syntax

> cat [OPTION]... [FILE]...

Options / Flags

Option / Flag	Meaning
-n	Number all output lines
-b	Number non-empty lines only
-s	Squeeze repeated blank lines
-A	Show non-printing characters
(no FILE)	Read stdin until Ctrl+D

Example

```
$ cat names.txt
$ cat -n project/README.md
$ cat names.txt scores.txt
```

- cat names.txt prints three names; cat -n project/README.md numbers the line.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ cat names.txt
Ali
Hodan
Yusuf
nasteha@ubuntu:~/linux-ch3-lab$ cat -n project/README.md
     1  Linux project
nasteha@ubuntu:~/linux-ch3-lab$ cat names.txt scores.txt
Ali
Hodan
Yusuf
42
100
7
```

Screenshot: cat (continued: -n, several files) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.2 seq

Purpose: Prints a sequence of numbers; used to build the 60-line long.txt.

Syntax

```
seq [OPTION]... LAST
seq FIRST LAST
```

Options / Flags

Option / Flag	Meaning
FIRST LAST	Range
-s SEP	Separator string
-w	Equalize width with zero padding

Example

```
$ seq 1 5
```

```
$ seq 1 60 > long.txt
$ wc -l long.txt
```

- seq 1 60 > long.txt creates the file for less and more.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ seq 1 5
1
2
3
4
5
nasteha@ubuntu:~/linux-ch3-lab$ seq 1 60 > long.txt
nasteha@ubuntu:~/linux-ch3-lab$ wc -l long.txt
60 long.txt
```

Screenshot: seq in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.3 less

Purpose: Pager: read long files one screen at a time, moving forward and backward and searching.

Syntax

```
less [OPTION]... FILE
```

Options / Flags

Option / Flag	Meaning
Space	Next page
b	Back a page
/word	Search forward
n	Next match
q	Quit (file unchanged)
-N	Show line numbers
-S	Do not wrap long lines

Example

```
$ less long.txt
```

- less long.txt then /40, n, q.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ less long.txt
1
2
3
4
5
6
7
```

```
8
9
10
...
long.txt (END) - press q to quit
```

Screenshot: less in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: less is an interactive full-screen pager (and is not installed in the build sandbox). Keys from the slide: Space, b, /40, n, q.

3.1.4 more

Purpose: Basic pager: shows a file one screen at a time (forward only).

Syntax

```
more [OPTION]... FILE
```

Options / Flags

Option / Flag	Meaning
Space	Advance one screen
Enter	Advance one line
q	Quit
-d	Show help prompts

Example

```
$ more long.txt
```

- more long.txt; use Space to advance and q to exit.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ more long.txt
1
2
3
...
--More--(38%)
```

Screenshot: more in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: more waits for keys in a full-screen display.

3.1.5 head

Purpose: Prints the first lines of a file (10 by default).

Syntax

```
head [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-n N	First N lines
-n -N	All but the last N lines
-c N	First N bytes
(default)	10 lines

Example

```
$ head -n 2 names.txt
$ head -n 3 long.txt
```

- head -n 2 names.txt shows the first two names.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ head -n 2 names.txt
Ali
Hodan
nasteha@ubuntu:~/linux-ch3-lab$ head -n 3 long.txt
1
2
3
```

Screenshot: head in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.6 tail

Purpose: Prints the last lines of a file (10 by default); -f follows new lines.

Syntax

```
tail [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-n N	Last N lines
-n +N	Start at line N
-c N	Last N bytes
-f	Follow: show new lines as they are appended (Ctrl+C stops)
(default)	10 lines

Example

```
$ tail -n 2 names.txt
```

- tail -n 2 names.txt shows the last two names.
- tail -n 2 -f app.log keeps running; here a second terminal appends 'INFO done'.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
```

```
nasteha@ubuntu:~/linux-ch3-lab$ tail -n 2 names.txt
Hodan
Yusuf
```

Screenshot: tail in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.7 tail -f (monitoring changes)

Purpose: Monitors a file in real time (log files).

Syntax

```
tail -n 2 -f FILE
```

Options / Flags

Option / Flag	Meaning
-n 2	Print two existing lines first
-f	Keep following
Ctrl + C	Stop monitoring

Example

```
$ tail -n 2 -f app.log
```

- Many systems use journalctl instead of /var/log/syslog.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ tail -n 2 -f app.log
WARN retry
ERROR login failed
[another terminal: echo 'INFO done' >> app.log]
INFO done
^C
```

Screenshot: tail -f (monitoring changes) in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: a following command never ends by itself, so it cannot be captured by a script.

3.1.8 file

Purpose: Identifies the real type of a file from its content (not its extension).

Syntax

```
file [OPTION]... FILE...
```

Options / Flags

Option / Flag	Meaning
-b	Brief: omit the file name
-i	Output MIME type
-L	Follow symlinks

Example

```
$ file names.txt
$ file project
$ file -i names.txt
```

- Linux inspects contents; an extension is only a label.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ file names.txt
names.txt: ASCII text
nasteha@ubuntu:~/linux-ch3-lab$ file project
project: directory
nasteha@ubuntu:~/linux-ch3-lab$ file -i names.txt
names.txt: text/plain; charset=us-ascii
```

Screenshot: file in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.9 stat

Purpose: Displays detailed file metadata: size, type, permissions, inode, times.

Syntax

```
stat [OPTION]... FILE...
```

Options / Flags

Option / Flag	Meaning
-c FORMAT	Print selected fields
-f	Show filesystem status
-L	Follow symlinks

Example

```
$ stat names.txt
```

- stat names.txt shows size 16 bytes, regular file, inode and times.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ stat names.txt
  File: names.txt
  Size: 16          Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 532608     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1002/ nasteha)   Gid: ( 1003/ nasteha)
Access: 2026-11-11 18:43:50.895673532 +0000
Modify: 2026-11-11 18:43:50.900904167 +0000
Change: 2026-11-11 18:43:50.900904167 +0000
 Birth: 2026-11-11 18:43:50.895673532 +0000
```

Screenshot: stat in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.10 ls -l (basic metadata)

Purpose: Basic file information: type, permissions, links, owner, size, date, name.

Syntax

```
ls -l FILE
```

Options / Flags

Option / Flag	Meaning
-l	Long listing
-h	Readable sizes
-i	Show inode numbers (Section 3.5)

Example

```
$ ls -l names.txt
```

- The three metadata commands are file, stat and ls -l.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ ls -l names.txt
-rw-r--r-- 1 nasteha nasteha 16 Nov 11 18:43 names.txt
```

Screenshot: ls -l (basic metadata) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.11 which

Purpose: Searches the PATH directories and prints the path of an executable.

Syntax

```
which [-a] COMMAND...
```

Options / Flags

Option / Flag	Meaning
-a	Print all matches, not only the first
(no output)	Not found, or a shell built-in like cd

Example

```
$ which ls
$ which cd
$ echo $?
```

- which ls prints /usr/bin/ls; which cd may print nothing.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ which ls
/usr/bin/ls
nasteha@ubuntu:~/linux-ch3-lab$ which cd
nasteha@ubuntu:~/linux-ch3-lab$ echo $?
```

```
0
```

Screenshot: which in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.12 type

Purpose: Shows whether a command is a built-in, an alias, a function or an executable file.

Syntax

```
type [-a] [-t] NAME...
```

Options / Flags

Option / Flag	Meaning
-a	Show all places
-t	Print one word: alias, builtin, file, function

Example

```
$ type cd
$ type ls
$ type -t cd
```

- type cd reports 'cd is a shell builtin'.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ type cd
cd is a shell builtin
nasteha@ubuntu:~/linux-ch3-lab$ type ls
ls is /usr/bin/ls
nasteha@ubuntu:~/linux-ch3-lab$ type -t cd
builtin
```

Screenshot: type in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.13 command -v

Purpose: Shows how the shell would resolve a command name.

Syntax

```
command -v NAME
```

Options / Flags

Option / Flag	Meaning
-v	Print the path/name used
-V	Verbose description

Example

```
$ command -v ls
$ command -v cd
```

- command -v ls resolves the command through the shell.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ command -v ls
/usr/bin/ls
nasteha@ubuntu:~/linux-ch3-lab$ command -v cd
cd
```

*Screenshot: command -v in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.1.14 whereis

Purpose: Locates the binary, source and manual-page files of a command.

Syntax

```
whereis [OPTION]... NAME...
```

Options / Flags

Option / Flag	Meaning
-b	Binaries only
-m	Manual pages only
-s	Sources only

Example

```
$ whereis tar
$ whereis -b tar
```

- whereis tar can identify program binaries and manual pages; locations vary.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ whereis tar
tar: /usr/bin/tar
nasteha@ubuntu:~/linux-ch3-lab$ whereis -b tar
tar: /usr/bin/tar
```

*Screenshot: whereis in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.1.15 find

Purpose: Searches directory trees live by name, type, size, time and more.

Syntax

```
find [PATH...] [EXPRESSION]
```

Options / Flags

Option / Flag	Meaning
-name 'PATTERN'	Match name (case-sensitive)

Option / Flag	Meaning
-iname 'PATTERN'	Match name ignoring case
-type f	Regular files
-type d	Directories
-size +10c	Larger than 10 bytes (c=bytes, k, M, G)
-mmin -30	Modified within the last 30 minutes
.	Search the current directory

Example

```
$ find . -name 'names.txt'
$ find . -name '*.log'
$ find project -type d
$ find . -name 'names.txt' -size +10c
$ find . -type f -mmin -30 | head -n 3
$ find project -name "*.py" -type f
$ find . -iname 'NAMES.TXT'
```

- The . path is the lab directory; quote patterns; tests combine with AND.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ find . -name 'names.txt'
./names.txt
nasteha@ubuntu:~/linux-ch3-lab$ find . -name '*.log'
./app.log
nasteha@ubuntu:~/linux-ch3-lab$ find project -type d
project
project/src
nasteha@ubuntu:~/linux-ch3-lab$ find . -name 'names.txt' -size +10c
./names.txt
nasteha@ubuntu:~/linux-ch3-lab$ find . -type f -mmin -30 | head -n 3
./names.txt
./scores.txt
./users.txt
nasteha@ubuntu:~/linux-ch3-lab$ find project -name "*.py" -type f
project/src/app.py
nasteha@ubuntu:~/linux-ch3-lab$ find . -iname 'NAMES.TXT'
./names.txt
```

Screenshot: find in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.16 locate

Purpose: Finds paths quickly in a prebuilt database (index).

Syntax

```
locate [OPTION]... PATTERN
```

Options / Flags

Option / Flag	Meaning
-i	Ignore case

Option / Flag	Meaning
-c	Count matches
-n N	Limit to N results
-e	Only existing files

Example

```
$ locate names.txt
$ touch recent.txt
$ locate recent.txt
$ find . -name 'recent.txt'
```

- A newly created file may be missing until the database is refreshed; find sees it at once.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ locate names.txt
[path appears only if indexed]
nasteha@ubuntu:~/linux-ch3-lab$ touch recent.txt
nasteha@ubuntu:~/linux-ch3-lab$ locate recent.txt
[may be missing from database]
nasteha@ubuntu:~/linux-ch3-lab$ find . -name 'recent.txt'
./recent.txt
```

Screenshot: locate in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: locate/plocate is not installed in the build sandbox and needs an index.

3.1.17 updatedb

Purpose: Rebuilds the database used by locate; usually needs administrator privileges.

Syntax

```
sudo updatedb
```

Options / Flags

Option / Flag	Meaning
sudo	Run as administrator
(no options)	Rescan the filesystem and refresh the index

Example

```
$ sudo updatedb
$ locate recent.txt
```

- Run it, then locate recent.txt finds the new file.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ sudo updatedb
[sudo] password for nasteha:
nasteha@ubuntu:~/linux-ch3-lab$ locate recent.txt
```

```
/home/nasteha/linux-ch3-lab/recent.txt
```

Screenshot: updatedb in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: requires sudo and the locate package.

3.1.18 grep

Purpose: Searches inside files and prints the lines that match a pattern (Global Regular Expression Print).

Syntax

```
grep [OPTION]... PATTERN [FILE]...
```

Options / Flags

Option / Flag	Meaning
-n	Show line numbers
-i	Ignore case
-v	Invert: show non-matching lines
-r	Recurse into directories
-R	Recurse and follow symlinks
-rn	Recursive with line numbers
-c	Count matching lines
-l	List only file names
-w	Match whole words

Example

```
$ grep 'Ali' names.txt
$ grep -n 'ERROR' app.log
$ grep -v 'ERROR' app.log
$ grep -rn 'TODO' project
$ grep -in 'error' app.log
```

- Place the pattern before the filename. grep searches content; find and locate search paths.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ grep 'Ali' names.txt
Ali
nasteha@ubuntu:~/linux-ch3-lab$ grep -n 'ERROR' app.log
2:ERROR disk full
4:ERROR login failed
nasteha@ubuntu:~/linux-ch3-lab$ grep -v 'ERROR' app.log
INFO start
WARN retry
nasteha@ubuntu:~/linux-ch3-lab$ grep -rn 'TODO' project
project/src/app.py:1:# TODO: test
nasteha@ubuntu:~/linux-ch3-lab$ grep -in 'error' app.log
2:ERROR disk full
4:ERROR login failed
```

Screenshot: grep in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.1.19 grep with regular expressions

Purpose: grep patterns can use regex symbols; prefer single quotes around patterns.

Syntax

```
grep 'PATTERN' FILE
```

Options / Flags

Option / Flag	Meaning
.	Any one character (b..h)
*	Previous character, zero or more times (lo*g)
^	Start of the line (^root)
$	End of the line (bash$)
[0-9]	Any one digit
[ae]	One of the listed characters (gr[ae]y)

Example

```
$ grep '^Ali' names.txt
$ grep 'failed$' app.log
$ grep 'gr[ae]y' words.txt
$ grep 'M.g' cities.txt
```

- ^Ali matches at line start; failed$ at line end; gr[ae]y matches gray and grey.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ grep '^Ali' names.txt
Ali
nasteha@ubuntu:~/linux-ch3-lab$ grep 'failed$' app.log
ERROR login failed
nasteha@ubuntu:~/linux-ch3-lab$ grep 'gr[ae]y' words.txt
gray
grey
nasteha@ubuntu:~/linux-ch3-lab$ grep 'M.g' cities.txt
Mogadishu
Mogadishu
Mogadishu
```

Screenshot: grep with regular expressions in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2 Redirection, Pipes, and Text Processing

Command input/output and combining small tools.

- Three standard streams: STDIN (fd 0, input), STDOUT (fd 1, normal results), STDERR (fd 2, errors).
- > creates/overwrites; >> appends; 2> redirects errors; 2>/dev/null discards errors (do not hide stderr while diagnosing); < redirects a file into stdin (it does not edit the file).

- | sends stdout of one command into stdin of the next (stderr is not piped by default). ; runs both commands; && runs the second only if the first succeeds; || only if it fails. Exit status 0 = success, non-zero = failure.

3.2.1 Standard streams: stdout vs stderr

Purpose: Shows normal output and error output being different streams.

Syntax

```
ls names.txt missing.txt
```

Options / Flags

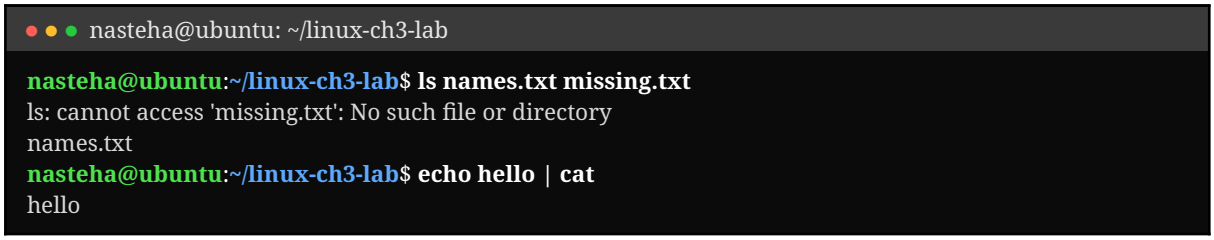
Option / Flag	Meaning
STDOUT (1)	names.txt - the valid path
STDERR (2)	the error for missing.txt
cat (no file)	Reads stdin until Ctrl+D

Example

```
$ ls names.txt missing.txt
$ echo hello | cat
```

- The valid path goes to stdout, the failure message to stderr; the order on screen can differ.

Terminal (screenshot of the command and its output)



Screenshot: Standard streams: stdout vs stderr in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.2 Output redirection > and >>

Purpose: Send stdout to a file: > overwrites, >> appends.

Syntax

```
COMMAND > FILE
COMMAND >> FILE
```

Options / Flags

Option / Flag	Meaning
>	Create or overwrite the file
>>	Append to the end
2>	Redirect stderr (below)

Example

```
$ printf 'First\n' > actions.txt
```

```
$ printf 'Second\n' >> actions.txt
$ cat actions.txt
```

- First creates actions.txt, second adds a line without erasing it. Check the target: > replaces data immediately.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ printf 'First\n' > actions.txt
nasteha@ubuntu:~/linux-ch3-lab$ printf 'Second\n' >> actions.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat actions.txt
First
Second
```

Screenshot: Output redirection > and >> in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.3 Error redirection 2>

Purpose: Redirects standard error (file descriptor 2) to a file.

Syntax

```
COMMAND 2> FILE
```

Options / Flags

Option / Flag	Meaning
2> FILE	Save error messages
> FILE	Save stdout
2>/dev/null	Discard errors
2>&1	Send stderr to the same place as stdout

Example

```
$ ls names.txt missing.txt > out.txt 2> err.txt
$ cat out.txt
$ cat err.txt
$ ls missing.txt 2>/dev/null
```

- ls names.txt missing.txt > out.txt 2> err.txt separates results from errors.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ ls names.txt missing.txt > out.txt 2> err.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat out.txt
names.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat err.txt
ls: cannot access 'missing.txt': No such file or directory
nasteha@ubuntu:~/linux-ch3-lab$ ls missing.txt 2>/dev/null
```

Screenshot: Error redirection 2> in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.4 Input redirection <

Purpose: Feeds a file into a command's standard input.

Syntax

```
COMMAND < FILE
```

Options / Flags

Option / Flag	Meaning
< FILE	Read stdin from FILE (no filename label)
wc -l names.txt	Prints count plus file name
wc -l < names.txt	Prints count only

Example

```
$ wc -l names.txt
$ wc -l < names.txt
$ tr 'a-z' 'A-Z' < names.txt
```

- < changes where the program reads from; the input file is not edited.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ wc -l names.txt
3 names.txt
nasteha@ubuntu:~/linux-ch3-lab$ wc -l < names.txt
3
nasteha@ubuntu:~/linux-ch3-lab$ tr 'a-z' 'A-Z' < names.txt
ALI
HODAN
YUSUF
```

Screenshot: Input redirection < in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.5 Pipes |

Purpose: Connects commands: stdout of the left becomes stdin of the right, no temporary file.

Syntax

```
COMMAND1 | COMMAND2 | ...
```

Options / Flags

Option / Flag	Meaning
|	Pass stdout only (fd 1)
Pipe vs >	| hands output to a program; > saves it to a file

Example

```
$ grep 'ERROR' app.log | wc -l
$ cat names.txt | wc -l
$ grep 'Mogadishu' cities.txt | wc -l
$ grep -i 'ERROR' app.log | sort | uniq -c
```

- Counts are known from the practice dataset: 2, 3, 3. Example from the slide: grep -i "ERROR" server.log | sort | uniq -c.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ grep 'ERROR' app.log | wc -l
2
nasteha@ubuntu:~/linux-ch3-lab$ cat names.txt | wc -l
3
nasteha@ubuntu:~/linux-ch3-lab$ grep 'Mogadishu' cities.txt | wc -l
3
nasteha@ubuntu:~/linux-ch3-lab$ grep -i 'ERROR' app.log | sort | uniq -c
      1 ERROR disk full
      1 ERROR login failed
```

*Screenshot: Pipes | in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.2.6 Command chaining ; && ||

Purpose: Conditional command chaining based on exit status.

Syntax

```
cmd1 ; cmd2
cmd1 && cmd2
cmd1 || cmd2
```

Options / Flags

Option / Flag	Meaning
;	Run both regardless of status
&&	Run the second only if the first succeeds
||	Run the second only if the first fails
$?	Exit status of the last command (0 = success)

Example

```
$ printf 'one\n'; printf 'two\n'
$ mkdir -p scratch && echo OK > scratch/status.txt
$ cat scratch/status.txt
$ test -f missing.txt || echo 'Not found'
```

- These control execution; unlike |, which passes data.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ printf 'one\n'; printf 'two\n'
one
two
nasteha@ubuntu:~/linux-ch3-lab$ mkdir -p scratch && echo OK > scratch/status.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat scratch/status.txt
OK
nasteha@ubuntu:~/linux-ch3-lab$ test -f missing.txt || echo 'Not found'
Not found
```

*Screenshot: Command chaining ; && || in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.2.7 test

Purpose: Evaluates a condition and sets the exit status (0 true, 1 false); used with && and ||.

Syntax

```
test EXPRESSION   (or [ EXPRESSION ])
```

Options / Flags

Option / Flag	Meaning
-f FILE	True if FILE is a regular file
-d FILE	True if a directory
-e FILE	True if it exists
-s FILE	True if size greater than zero

Example

```
$ test -f names.txt && echo exists
$ test -f missing.txt || echo 'Not found'
$ test -d project; echo $?
```

- test -f missing.txt || echo 'Not found' prints Not found.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ test -f names.txt && echo exists
exists
nasteha@ubuntu:~/linux-ch3-lab$ test -f missing.txt || echo 'Not found'
Not found
nasteha@ubuntu:~/linux-ch3-lab$ test -d project; echo $?
0
```

Screenshot: test in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.8 sort

Purpose: Sorts lines of text (alphabetical by default).

Syntax

```
sort [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-n	Numeric sort
-r	Reverse order
-rn	Numeric, largest first
-u	Unique lines only
-k N	Sort by field N
-t SEP	Field separator

Example

```
$ sort scores.txt
$ sort -n scores.txt
$ sort -rn scores.txt
```

- Without -n, 100 appears before 7 as text; -n orders 7, 42, 100.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ sort scores.txt
100
42
7
nasteha@ubuntu:~/linux-ch3-lab$ sort -n scores.txt
7
42
100
nasteha@ubuntu:~/linux-ch3-lab$ sort -rn scores.txt
100
42
7
```

Screenshot: sort in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.9 uniq

Purpose: Collapses repeated adjacent lines; sort first to group duplicates.

Syntax

```
uniq [OPTION]... [INPUT]
```

Options / Flags

Option / Flag	Meaning
-c	Prefix lines with their count
-d	Show only duplicated lines
-u	Show only unique lines
-i	Ignore case

Example

```
$ sort cities.txt | uniq
$ sort cities.txt | uniq -c
$ sort cities.txt | uniq -d
```

- uniq alone cannot remove non-adjacent duplicates, so use sort | uniq.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ sort cities.txt | uniq
Garowe
Hargeisa
```

```
Mogadishu
nasteha@ubuntu:~/linux-ch3-lab$ sort cities.txt | uniq -c
      1 Garowe
      1 Hargeisa
      3 Mogadishu
nasteha@ubuntu:~/linux-ch3-lab$ sort cities.txt | uniq -d
Mogadishu
```

Screenshot: uniq in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.10 cut

Purpose: Extracts characters, bytes or fields from each line.

Syntax

```
cut OPTION... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-d DELIM	Field delimiter (here :)
-f LIST	Fields to keep (1 or 1,3)
-c RANGE	Characters (1-4)
-b LIST	Bytes

Example

```
$ cut -d: -f1 users.txt
$ cut -d: -f 1,3 users.txt
$ echo "somalia" | cut -c 1-4
```

- With -f, give the correct delimiter or the fields will not split.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ cut -d: -f1 users.txt
ali
hodan
yusuf
nasteha@ubuntu:~/linux-ch3-lab$ cut -d: -f 1,3 users.txt
ali:bash
hodan:sh
yusuf:bash
nasteha@ubuntu:~/linux-ch3-lab$ echo "somalia" | cut -c 1-4
soma
```

Screenshot: cut in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.11 paste

Purpose: Combines corresponding lines of files side by side (tab separated by default).

Syntax

```
paste [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-d DELIM	Use DELIM instead of tab
-s	Serial: all lines of a file into one row

Example

```
$ paste names.txt scores.txt
$ paste -d, names.txt scores.txt
$ paste -s -d, names.txt
```

- Files with different line counts give blank cells for missing values.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ paste names.txt scores.txt
Ali     42
Hodan   100
Yusuf   7
nasteha@ubuntu:~/linux-ch3-lab$ paste -d, names.txt scores.txt
Ali,42
Hodan,100
Yusuf,7
nasteha@ubuntu:~/linux-ch3-lab$ paste -s -d, names.txt
Ali,Hodan,Yusuf
```

Screenshot: paste in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.12 wc

Purpose: Counts lines, words and bytes.

Syntax

```
wc [OPTION]... [FILE]...
```

Options / Flags

Option / Flag	Meaning
-l	Lines (newline characters)
-w	Words
-c	Bytes
-m	Characters

Example

```
$ wc -l names.txt
```

```
$ grep -i 'error' app.log | wc -l
$ wc -w notes.txt
$ wc names.txt
```

- Use wc -l when the question is the number of matching lines.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ wc -l names.txt
3 names.txt
nasteha@ubuntu:~/linux-ch3-lab$ grep -i 'error' app.log | wc -l
2
nasteha@ubuntu:~/linux-ch3-lab$ wc -w notes.txt
6 notes.txt
nasteha@ubuntu:~/linux-ch3-lab$ wc names.txt
 3  3 16 names.txt
```

*Screenshot: wc in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.2.13 tr

Purpose: Translates, deletes or squeezes individual characters; reads only stdin (use < or |).

Syntax

```
tr [OPTION]... SET1 [SET2]
```

Options / Flags

Option / Flag	Meaning
SET1 SET2	Translate (a-z to A-Z)
-d	Delete characters
-s	Squeeze repeated characters
-c	Complement the set
[:alpha:] [:upper:] [:lower:]	Character classes

Example

```
$ tr 'a-z' 'A-Z' < names.txt
$ echo 'too    many' | tr -s ' '
$ echo 'a1b2c3' | tr -d '0-9'
```

- tr does not accept a filename.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ tr 'a-z' 'A-Z' < names.txt
ALI
HODAN
YUSUF
nasteha@ubuntu:~/linux-ch3-lab$ echo 'too    many' | tr -s ' '
too many
nasteha@ubuntu:~/linux-ch3-lab$ echo 'a1b2c3' | tr -d '0-9'
abc
```

*Screenshot: tr in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)***

3.2.14 Pipeline: login shells

Purpose: Combine cut, sort and uniq to count login shells.

Syntax

```
cut -d: -f3 users.txt | sort | uniq -c
```

Options / Flags

Option / Flag	Meaning
cut	Isolate field 3 (the shell)
sort	Group identical shells
uniq -c	Count; add sort -nr for highest first

Example

```
$ cut -d: -f3 users.txt | sort
$ cut -d: -f3 users.txt | sort | uniq -c
```

- The sample has two bash entries and one sh entry.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ cut -d: -f3 users.txt | sort
bash
bash
sh
nasteha@ubuntu:~/linux-ch3-lab$ cut -d: -f3 users.txt | sort | uniq -c
      2 bash
      1 sh
```

Screenshot: Pipeline: login shells in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.15 Pipeline: log messages

Purpose: Search for case-insensitive error text and count the matches.

Syntax

```
grep -i 'error' app.log | wc -l
```

Options / Flags

Option / Flag	Meaning
grep -i	Match error in any capitalization
wc -l	Count matching lines

Example

```
$ grep -i 'error' app.log
$ grep -i 'error' app.log | wc -l
```

- The demo log has exactly two ERROR records; no file is changed.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ grep -i 'error' app.log
ERROR disk full
ERROR login failed
nasteha@ubuntu:~/linux-ch3-lab$ grep -i 'error' app.log | wc -l
2
```

Screenshot: Pipeline: log messages in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.2.16 Pipeline: common words

Purpose: Split a text into words, normalize case and count frequencies.

Syntax

```
tr -cs '[:alpha:]' '\n' < notes.txt | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -n 1
```

Options / Flags

Option / Flag	Meaning
tr -cs '[:alpha:]' '\n'	Split on any non-letter
tr '[:upper:]' '[:lower:]'	Lowercase everything
sort | uniq -c	Count
sort -nr | head -n 1	Top result

Example

```
$ tr -cs '[:alpha:]' '\n' < notes.txt | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -n 1
```

- The result for notes.txt is 2 linux.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ tr -cs '[:alpha:]' '\n' < notes.txt | tr '[:upper:]' '[:lower:]' | sort | uniq -c | sort -nr | head -n 1
      2 linux
```

Screenshot: Pipeline: common words in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3 Archives and Compression

Archiving combines files into one file (no size reduction necessarily); compression reduces size. tar = Tape Archive.

- Formats: .tar bundles; .tar.gz widely used on Linux; .tar.xz often smaller but slower; .zip for cross-platform exchange (Windows and macOS open it natively).
- Inspect an archive (tar -t, unzip -l) before extracting; extract untrusted archives into a controlled folder. For directories use tar with gzip, not plain gzip.

tar option	Meaning
-c	Create a new archive
-x	Extract files
-v	Display processed filenames
-f	Specify the archive filename
-t	List contents
-z	gzip compression
-j	bzip2 compression
-J	xz compression
-C	Change to a specified directory

3.3.1 tar

Purpose: Creates, lists and extracts archives, optionally compressed.

Syntax

```
tar [OPTIONS] -f ARCHIVE [FILES...]
```

Options / Flags

Option / Flag	Meaning
-c	Create
-x	Extract
-t	List contents
-v	Verbose
-f FILE	Archive file name (right after -f)
-z	gzip (.tar.gz)
-j	bzip2 (.tar.bz2)
-J	xz (.tar.xz); uppercase J differs from j
-C DIR	Change to / extract into DIR

Example

```
$ tar -cf notes.tar notes.txt words.txt
$ tar -cvf project.tar project/
$ tar -tf project.tar
$ mkdir -p restored-tar
$ tar -xf project.tar -C restored-tar
$ ls restored-tar
$ tar -czf project.tar.gz project/
$ tar -cjf project.tar.bz2 project/
$ tar -cJf project.tar.xz project/
$ tar -tzf project.tar.gz
$ ls -l project.tar*
```

- Create, list, extract into restored-tar/, then compressed variants.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ tar -cf notes.tar notes.txt words.txt
nasteha@ubuntu:~/linux-ch3-lab$ tar -cvf project.tar project/
project/
project/src/
project/src/app.py
project/README.md
project/config.txt
nasteha@ubuntu:~/linux-ch3-lab$ tar -tf project.tar
project/
project/src/
project/src/app.py
project/README.md
project/config.txt
nasteha@ubuntu:~/linux-ch3-lab$ mkdir -p restored-tar
nasteha@ubuntu:~/linux-ch3-lab$ tar -xf project.tar -C restored-tar
nasteha@ubuntu:~/linux-ch3-lab$ ls restored-tar
project
nasteha@ubuntu:~/linux-ch3-lab$ tar -czf project.tar.gz project/
nasteha@ubuntu:~/linux-ch3-lab$ tar -cjf project.tar.bz2 project/
nasteha@ubuntu:~/linux-ch3-lab$ tar -cJf project.tar.xz project/
nasteha@ubuntu:~/linux-ch3-lab$ tar -tzf project.tar.gz
project/
project/src/
project/src/app.py
project/README.md
project/config.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls -l project.tar*
-rw-r--r-- 1 nasteha nasteha 10240 Nov 11 18:43 project.tar
-rw-r--r-- 1 nasteha nasteha   284 Nov 11 18:43 project.tar.bz2
-rw-r--r-- 1 nasteha nasteha   268 Nov 11 18:43 project.tar.gz
-rw-r--r-- 1 nasteha nasteha   292 Nov 11 18:43 project.tar.xz
```

Screenshot: tar in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.2 gzip

Purpose: Compresses a single file (.gz); removes the source unless -k.

Syntax

```
gzip [OPTION]... FILE...
```

Options / Flags

Option / Flag	Meaning
-k	Keep the original
-9	Best compression, slowest
-1	Fastest
-d	Decompress (= gunzip)
-c	Write to stdout

Example

```
$ gzip -k -9 notes.txt
```

```
$ ls notes.txt*
```

- gzip -k -9 notes.txt keeps notes.txt and creates notes.txt.gz.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ gzip -k -9 notes.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls notes.txt*
notes.txt  notes.txt.gz
```

Screenshot: gzip in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.3 gunzip

Purpose: Decompresses .gz files.

Syntax

```
gunzip [OPTION]... FILE.gz
```

Options / Flags

Option / Flag	Meaning
-c	Print restored contents without removing the .gz
-k	Keep the .gz file

Example

```
$ gunzip -c notes.txt.gz
$ ls notes.txt*
```

- gunzip -c notes.txt.gz prints the original text.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ gunzip -c notes.txt.gz
Linux tools help. Linux is useful.
nasteha@ubuntu:~/linux-ch3-lab$ ls notes.txt*
notes.txt  notes.txt.gz
```

Screenshot: gunzip in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.4 bzip2

Purpose: Compresses a file with bzip2 (.bz2); -k keeps the original.

Syntax

```
bzip2 [OPTION]... FILE
```

Options / Flags

Option / Flag	Meaning
-k	Keep input file
-d	Decompress

Option / Flag	Meaning
-9	Best compression
-z	Compress

Example

```
$ bzip2 -k app.log
$ ls app.log*
```

- bzip2 -k app.log creates app.log.bz2.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ bzip2 -k app.log
nasteha@ubuntu:~/linux-ch3-lab$ ls app.log*
app.log  app.log.bz2
```

Screenshot: bzip2 in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.5 bzcat

Purpose: Prints a .bz2 file's decompressed contents without unpacking it.

Syntax

```
bzcat FILE.bz2
```

Options / Flags

Option / Flag	Meaning
(none)	Decompress to stdout

Example

```
$ bzcat app.log.bz2 | head -n 1
```

- bzcat app.log.bz2 | head -n 1 shows INFO start.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ bzcat app.log.bz2 | head -n 1
INFO start
```

Screenshot: bzcat in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.6 bunzip2

Purpose: Decompresses .bz2 files.

Syntax

```
bunzip2 [OPTION]... FILE.bz2
```

Options / Flags

Option / Flag	Meaning
-k	Keep the .bz2
-c	To stdout

Example

```
$ bunzip2 -k app.log.bz2
$ ls app.log*
```

- Unpacks the compressed copy.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ bunzip2 -k app.log.bz2
nasteha@ubuntu:~/linux-ch3-lab$ ls app.log*
app.log  app.log.bz2
```

Screenshot: bunzip2 in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.7 xz

Purpose: Compresses a file with xz (.xz); often smaller but slower.

Syntax

```
xz [OPTION]... FILE
```

Options / Flags

Option / Flag	Meaning
-k	Keep input file
-d	Decompress
-9	Best compression
-T N	Use N threads

Example

```
$ xz -k app.log
$ ls app.log*
```

- xz -k app.log creates app.log.xz.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ xz -k app.log
nasteha@ubuntu:~/linux-ch3-lab$ ls app.log*
app.log  app.log.xz
```

Screenshot: xz in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.8 xzcat

Purpose: Prints a .xz file's contents without unpacking it.

Syntax

```
xzcat FILE.xz
```

Options / Flags

Option / Flag	Meaning
(none)	Decompress to stdout

Example

```
$ xzcat app.log.xz | head -n 1
```

- xzcat app.log.xz | head -n 1 shows INFO start.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ xzcat app.log.xz | head -n 1
INFO start
```

Screenshot: xzcat in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.9 unxz

Purpose: Decompresses .xz files.

Syntax

```
unxz [OPTION]... FILE.xz
```

Options / Flags

Option / Flag	Meaning
-k	Keep the .xz
-c	To stdout

Example

```
$ unxz -k app.log.xz
$ ls app.log*
```

- Unpacks the compressed copy.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ unxz -k app.log.xz
nasteha@ubuntu:~/linux-ch3-lab$ ls app.log*
app.log  app.log.xz
```

Screenshot: unxz in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.10 zip

Purpose: Creates ZIP archives that open on Linux, Windows and macOS.

Syntax

```
zip [OPTION]... ARCHIVE.zip FILE...
```

Options / Flags

Option / Flag	Meaning
-r	Recurse into directories
-q	Quiet
-9	Best compression
-e	Encrypt with a password

Example

```
$ zip -r project.zip project/
```

- zip -r project.zip project/ adds the directory tree.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ zip -r project.zip project/
  adding: project/ (stored 0%)
  adding: project/src/ (stored 0%)
  adding: project/src/app.py (stored 0%)
  adding: project/README.md (stored 0%)
  adding: project/config.txt (stored 0%)
```

Screenshot: zip in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.3.11 unzip

Purpose: Lists or extracts ZIP archives.

Syntax

```
unzip [OPTION]... ARCHIVE.zip
```

Options / Flags

Option / Flag	Meaning
-l	List entries without extracting
-d DIR	Extract into DIR
-o	Overwrite without asking
-t	Test the archive

Example

```
$ unzip -l project.zip
$ unzip project.zip -d restored-zip
$ ls restored-zip
```

- Preview first with -l, then extract into an empty folder with -d.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ unzip -l project.zip
Archive:  project.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-11-11 18:43   project/
        0  2026-11-11 18:43   project/src/
       13  2026-11-11 18:43   project/src/app.py
       14  2026-11-11 18:43   project/README.md
       14  2026-11-11 18:43   project/config.txt
---------                     -------
       41                     5 files
nasteha@ubuntu:~/linux-ch3-lab$ unzip project.zip -d restored-zip
Archive:  project.zip
   creating: restored-zip/project/
   creating: restored-zip/project/src/
  extracting: restored-zip/project/src/app.py
  extracting: restored-zip/project/README.md
  extracting: restored-zip/project/config.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls restored-zip
project
```

Screenshot: unzip in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.4 Terminal Text Editors: Nano and Vim

Edit text files directly in a terminal, including when no graphical desktop is available.

- Nano: beginner-friendly; shortcut hints at the bottom (^ = Ctrl). Ctrl+O save (Write Out, Enter to confirm), Ctrl+X exit, Ctrl+W search, Ctrl+K cut line, Ctrl+U paste, Alt+U undo, Ctrl+G help, Ctrl+R read file, Ctrl+\ replace.
- Vim has three modes: Normal (start; keys are commands), Insert (press i; type text), Command line (press : for :w :q :wq :q!). Esc returns to Normal. If keystrokes delete text, press Esc and check the mode.
- Use project/config.txt for exercises, not a privileged system config; save a practice copy first.

3.4.1 nano

Purpose: Beginner-friendly terminal text editor.

Syntax

```
nano [OPTION]... [FILE]
```

Options / Flags

Option / Flag	Meaning
Ctrl+O	Write Out (save); Enter confirms the name
Ctrl+X	Exit (asks to save if changed)
Ctrl+W	Where Is (search)
Ctrl+K	Cut the current line

Option / Flag	Meaning
Ctrl+U	Paste (uncut)
Alt+U	Undo
Ctrl+G	Help
Ctrl+R	Read a file into the buffer
Ctrl+	Replace

Example

```
$ nano project/README.md
$ cat project/README.md
```

- nano project/README.md, add the line 'Practicing Nano', Ctrl+O, Enter, Ctrl+X, then cat the file.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ nano project/README.md
  GNU nano 7.2         project/README.md
Linux project
Practicing Nano
^G Help   ^O Write Out  ^W Where Is  ^K Cut   ^X Exit
^R Read File ^\ Replace  ^U Paste
[press Ctrl+O, Enter, then Ctrl+X]
nasteha@ubuntu:~/linux-ch3-lab$ cat project/README.md
Linux project
Practicing Nano
```

Screenshot: nano in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: nano is a full-screen interactive editor (not installed in the build sandbox).

3.4.2 vim

Purpose: Modal terminal editor. Normal mode first; i inserts; Esc returns; : opens the command line.

Syntax

```
vim [OPTION]... [FILE]
```

Options / Flags

Option / Flag	Meaning
i	Insert mode
Esc	Back to Normal mode
:w	Save
:q	Quit (refuses if unsaved changes: E37)
:wq	Save and quit
:q!	Quit and discard changes

Example

```
$ vim vim-practice.txt
```

```
$ cat vim-practice.txt
$ vim project/config.txt
```

- vim vim-practice.txt: press i, type 'Vim practice', Esc, :wq Enter, then cat the file.
- vim project/config.txt: change, Esc, :q gives E37, :q! discards.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ vim vim-practice.txt
~
~
"vim-practice.txt" [New]
[press i; type: Vim practice]
[press Esc; type :wq then Enter]
nasteha@ubuntu:~/linux-ch3-lab$ cat vim-practice.txt
Vim practice
nasteha@ubuntu:~/linux-ch3-lab$ vim project/config.txt
[press i; make a change; press Esc]
[type :q then Enter]
E37: No write since last change (add ! to override)
[type :q! then Enter]
```

Screenshot: vim in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: vim is a full-screen interactive editor (not installed in the build sandbox).

3.4.3 vi

Purpose: On Ubuntu, vi opens the same editor as vim (Vim in compatible mode).

Syntax

```
vi [FILE]
```

Options / Flags

Option / Flag	Meaning
(same keys as vim)	i, Esc, :w, :q, :wq, :q!

Example

```
$ vi hello.txt
```

- vi hello.txt starts in Normal mode with ~ on empty lines.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ vi hello.txt
~
~
"hello.txt" [New]
```

Screenshot: vi in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: interactive editor.

3.4.4 vimtutor

Purpose: A built-in interactive Vim lesson of about 30 minutes.

Syntax

```
vimtutor [LANGUAGE]
```

Options / Flags

Option / Flag	Meaning
(none)	Start the tutorial in English
:q	Leave the tutor

Example

```
$ vimtutor
```

- Run it for more Vim practice.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab
nasteha@ubuntu:~/linux-ch3-lab$ vimtutor
===============================================================================
=  W e l c o m e  t o  t h e  V I M  T u t o r  -  Version 1.7   =
===============================================================================
```

Screenshot: vimtutor in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: interactive tutorial.

3.5 Hard Links and Symbolic Links

A hard link is a second directory entry pointing to the same inode; a symbolic link is a small file storing a path.

- Hard link: two names, one inode, equal names, survives removal of the other name; same filesystem; not for directories.
- Symbolic link: own inode, stores the target path, can point to directories or other filesystems, becomes dangling if the target moves. Relative targets are resolved from the link's directory.
- Inspect with ls -li file hardlink and ls -l symlink.

3.5.1 ln

Purpose: Creates a hard link: another name for the same inode and data.

Syntax

```
ln [OPTION]... TARGET LINK_NAME
```

Options / Flags

Option / Flag	Meaning
(none)	Hard link
-s	Symbolic link (below)

Option / Flag	Meaning
-f	Remove existing destination
-v	Verbose

Example

```
$ cp notes.txt link-source.txt
$ ln link-source.txt link-hard.txt
$ ls -li link-source.txt link-hard.txt
$ echo "added" >> link-hard.txt
$ tail -n 1 link-source.txt
```

- Copy notes.txt, link it, write through one name and read from the other.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ cp notes.txt link-source.txt
nasteha@ubuntu:~/linux-ch3-lab$ ln link-source.txt link-hard.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls -li link-source.txt link-hard.txt
532662 -rw-r--r-- 2 nasteha nasteha 35 Nov 11 18:43 link-hard.txt
532662 -rw-r--r-- 2 nasteha nasteha 35 Nov 11 18:43 link-source.txt
nasteha@ubuntu:~/linux-ch3-lab$ echo "added" >> link-hard.txt
nasteha@ubuntu:~/linux-ch3-lab$ tail -n 1 link-source.txt
added
```

Screenshot: ln in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.5.2 ln -s

Purpose: Creates a symbolic (soft) link that stores a path.

Syntax

```
ln -s TARGET LINK_NAME
```

Options / Flags

Option / Flag	Meaning
-s	Symbolic link
ls -l	Shows link -> target
Dangling	Target renamed or removed

Example

```
$ cp notes.txt link-source.txt
$ ln -s link-source.txt link-shortcut.txt
$ ls -l link-shortcut.txt
$ cat link-shortcut.txt
$ mv link-source.txt link-renamed.txt
$ cat link-shortcut.txt
```

- After renaming the source, the stored path no longer resolves.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ cp notes.txt link-source.txt
nasteha@ubuntu:~/linux-ch3-lab$ ln -s link-source.txt link-shortcut.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls -l link-shortcut.txt
lrwxrwxrwx 1 nasteha nasteha 15 Nov 11 18:43 link-shortcut.txt -> link-source.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat link-shortcut.txt
Linux tools help. Linux is useful.
nasteha@ubuntu:~/linux-ch3-lab$ mv link-source.txt link-renamed.txt
nasteha@ubuntu:~/linux-ch3-lab$ cat link-shortcut.txt
cat: link-shortcut.txt: No such file or directory
```

Screenshot: ln -s in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

3.5.3 ls -li

Purpose: Lists inode numbers (-i) with the long format to compare hard links and symlinks.

Syntax

```
ls -li FILE...
```

Options / Flags

Option / Flag	Meaning
-i	Show the inode number
-l	Long listing; second column = link count

Example

```
$ cp notes.txt link-source.txt
$ ln link-source.txt link-hard.txt
$ ln -s link-source.txt link-soft.txt
$ ls -li link-source.txt link-hard.txt link-soft.txt
```

- Hard-link names share one inode and show link count 2; a symlink has its own inode.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~/linux-ch3-lab

nasteha@ubuntu:~/linux-ch3-lab$ cp notes.txt link-source.txt
nasteha@ubuntu:~/linux-ch3-lab$ ln link-source.txt link-hard.txt
nasteha@ubuntu:~/linux-ch3-lab$ ln -s link-source.txt link-soft.txt
nasteha@ubuntu:~/linux-ch3-lab$ ls -li link-source.txt link-hard.txt link-soft.txt
532662 -rw-r--r-- 2 nasteha nasteha 35 Nov 11 18:43 link-hard.txt
532663 lrwxrwxrwx 1 nasteha nasteha 15 Nov 11 18:43 link-soft.txt -> link-source.txt
532662 -rw-r--r-- 2 nasteha nasteha 35 Nov 11 18:43 link-source.txt
```

Screenshot: ls -li in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

Chapter 4: Process, Service, and System Management

Chapter 4 overview - commands in this chapter

Command / topic	Section	No.	Output
echo $$	Processes and Jobs	4.1.1	Real
ps	Processes and Jobs	4.1.2	Representative
pstree	Processes and Jobs	4.1.3	Representative
top	Processes and Jobs	4.1.4	Representative
htop	Processes and Jobs	4.1.5	Representative
sleep	Processes and Jobs	4.1.6	Representative
Ctrl+C and Ctrl+Z	Processes and Jobs	4.1.7	Representative
jobs	Processes and Jobs	4.1.8	Representative
bg	Processes and Jobs	4.1.9	Representative
fg	Processes and Jobs	4.1.10	Representative
kill -l	Processes and Jobs	4.1.11	Real
kill	Signals, kill, and Priority	4.2.1	Representative
pgrep	Signals, kill, and Priority	4.2.2	Representative
pkill	Signals, kill, and Priority	4.2.3	Representative
killall	Signals, kill, and Priority	4.2.4	Representative
Live example: start, find, stop	Signals, kill, and Priority	4.2.5	Representative
nice	Signals, kill, and Priority	4.2.6	Representative
renice	Signals, kill, and Priority	4.2.7	Representative
sudo (kill, renice)	Signals, kill, and Priority	4.2.8	Representative
uname (continued: -m, -n, -a)	System Information	4.3.1	Real
hostnamectl	System Information	4.3.2	Representative
uptime	System Information	4.3.3	Real
nproc	System Information	4.3.4	Real
free	System Information	4.3.5	Real
lscpu	System Information	4.3.6	Real
lsblk	System Information	4.3.7	Representative
/proc files	System Information	4.3.8	Real
/sys files	System Information	4.3.9	Real
systemctl cat	Services with systemd	4.4.1	Representative
systemctl status	Services with systemd	4.4.2	Representative
systemctl start / stop / restart	Services with systemd	4.4.3	Representative
systemctl reload	Services with systemd	4.4.4	Representative
systemctl enable / disable	Services with systemd	4.4.5	Representative
systemctl list-units	Services with systemd	4.4.6	Representative

Command / topic	Section	No.	Output
journalctl	Logs, Boot, and First Troubleshooting	4.5.1	Representative
journalctl -xe and -f	Logs, Boot, and First Troubleshooting	4.5.2	Representative
logger	Logs, Boot, and First Troubleshooting	4.5.3	Representative
/var/log (less, tail, ls)	Logs, Boot, and First Troubleshooting	4.5.4	Real
auth.log, journalctl _COMM=sudo, last	Logs, Boot, and First Troubleshooting	4.5.5	Representative
dmesg	Logs, Boot, and First Troubleshooting	4.5.6	Representative
shutdown	Logs, Boot, and First Troubleshooting	4.5.7	Representative
reboot	Logs, Boot, and First Troubleshooting	4.5.8	Representative
poweroff	Logs, Boot, and First Troubleshooting	4.5.9	Representative
sshd -t	Logs, Boot, and First Troubleshooting	4.5.10	Representative
Slow-machine order: uptime, free -h, top	Logs, Boot, and First Troubleshooting	4.5.11	Representative

4.1 Processes and Jobs

Every command you type becomes a process. The kernel gives each a PID; the process that started it is its parent (PPID).

- Family: systemd (PID 1, started by the kernel) > gnome-terminal > bash (your shell) > the command you typed. $$ holds the PID of your shell.
- ps is a snapshot (no refresh); top redraws every few seconds (P sort by CPU, M by memory, k kill, r renice, q quit); htop is a friendlier top if installed (F6 sort, F9 signal, F10/q quit). Names in [brackets] in ps output are kernel threads.
- STAT: R running, S sleeping, T stopped, Z zombie, + foreground. Foreground job holds the terminal; command & runs in the background and prints [job] PID.
- Ctrl+C sends SIGINT (job ends); Ctrl+Z sends SIGTSTP (job paused/Stopped). jobs lists, bg %N resumes in background, fg %N brings to front. Jobs belong to one terminal.
- Note: in a real interactive terminal the shell prints the job line '[1] PID' when you use &; transcripts below that were captured by a script do not show that line, and job-control transcripts are marked representative.

4.1.1 echo $$

Purpose: $$ is the PID of the shell you are typing in; ps shows that its children have it as PPID.

Syntax

```
echo $$
```

Options / Flags

Option / Flag	Meaning
$$	PID of the current shell
$PPID	PID of the parent

Example

```
$ echo $$
$ ps -o pid,ppid,comm
```

- The PID differs on every machine and every terminal (slide example: 2318).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ echo $$
2481
nasteha@ubuntu:~$ ps -o pid,ppid,comm
  PID  PPID COMMAND
 2481  2480 bash
 2483  2481 ps
```

Screenshot: echo $$ in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.1.2 ps

Purpose: Shows a snapshot of processes.

Syntax

```
ps [options]
```

Options / Flags

Option / Flag	Meaning
(none)	Processes in this terminal
-f	Full format: owner, PPID, start time
-p PID	Only this process
-o COLS	Only the columns you name (pid,ppid,comm,user,stat,cmd,ni)
-e	Every process
-ef	Every process, full format
aux	Every process of every user with %CPU and %MEM
--sort=-%mem	Largest memory users first

Example

```
$ ps
$ ps -f
$ ps -p 1 -o pid,user,comm
$ ps aux | less
$ ps aux --sort=-%mem | head -n 5
$ ps -ef | head -n 4
```

- ps -p 1 -o pid,comm shows PID 1; ps aux and ps -ef list everything (paged with less or limited with head).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ps
    PID TTY          TIME CMD
   2318 pts/0    00:00:00 bash
   2460 pts/0    00:00:00 ps
nasteha@ubuntu:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
nasteha     2318    2301  0 08:38 pts/0    00:00:00 bash
nasteha     2461    2318  0 08:46 pts/0    00:00:00 ps -f
nasteha@ubuntu:~$ ps -p 1 -o pid,user,comm
    PID USER     COMMAND
      1 root     systemd
nasteha@ubuntu:~$ ps aux | less
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.3  23380 13824 ?        Ss   08:02   0:02 /sbin/init splash
root         812  0.0  0.0   7196  2816 ?        Ss   08:02   0:00 /usr/sbin/cron -f -P
nasteha     2318  0.0  0.1  11504  5376 pts/0    Ss   08:38   0:00 bash
nasteha@ubuntu:~$ ps aux --sort=-%mem | head -n 5
nasteha@ubuntu:~$ ps -ef | head -n 4
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 08:02 ?        00:00:02 /sbin/init
root           2       0  0 08:02 ?        00:00:00 [kthreadd]
root           3       2  0 08:02 ?        00:00:00 [rcu_gp]
```

Screenshot: ps in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.1.3 pstree

Purpose: Draws the process family as a tree.

Syntax

```
pstree [-p] [PID]
```

Options / Flags

Option / Flag	Meaning
-p	Show PIDs in brackets
PID	Start the tree at this process
$$	Start at your shell

Example

```
$ pstree -p $$
```

- pstree -p $$ shows bash as the parent of pstree.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ pstree -p $$
bash(2318)──pstree(2530)
```

Screenshot: pstree in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: pstree (psmisc package) is not installed in the build sandbox; PIDs vary.

4.1.4 top

Purpose: Live, refreshing view of processes, busiest first.

Syntax

```
top [options]
```

Options / Flags

Option / Flag	Meaning
-u USER	Only that user's processes
-p PID	Watch one process
-b	Batch mode (non-interactive, for scripts)
-n N	N iterations then exit
P / M	Sort by CPU / memory
k / r	Kill / renice
q	Quit

Example

```
$ top
$ top -u nasteha
```

- Reading it: line 1 = clock, uptime, users, load average; Tasks; %Cpu(s) (id = idle); memory; then the process table. Here one batch snapshot is captured.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ top
top - 10:43:07 up  2:41,  1 user,  load average: 0.08, 0.12, 0.10
Tasks: 231 total,   1 running, 230 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.3 us,  0.7 sy,  0.0 ni, 97.8 id,  0.0 wa,  0.0 hi,  0.2 si,  0.0 st
MiB Mem :   3916.0 total,   1287.4 free,   1402.6 used,   1504.2 buff/cache
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1873 nasteha   20   0 4720312 352140 128804 S   2.3   8.8   1:12.40 gnome-shell
nasteha@ubuntu:~$ top -u nasteha
```

Screenshot: top in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.1.5 htop

Purpose: Friendlier top with colour bars, mouse and function keys, if installed (Ubuntu Server usually has it).

Syntax

```
htop [options]
```

Options / Flags

Option / Flag	Meaning
-u USER	Only that user's processes
-t	Tree view
F6	Choose sort column
F9	Send a signal
F10, q	Quit

Example

```
$ command -v htop
$ htop
$ htop -u nasteha
$ htop -t
```

- command -v htop tells whether it is installed; no output means use top.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ command -v htop
/usr/bin/htop
nasteha@ubuntu:~$ htop
nasteha@ubuntu:~$ htop -u nasteha
nasteha@ubuntu:~$ htop -t
```

Screenshot: htop in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: htop is a full-screen program and is not installed in the build sandbox.

4.1.6 sleep

Purpose: Waits for a number of seconds; used to create harmless foreground and background jobs.

Syntax

```
sleep NUMBER[SUFFIX]
```

Options / Flags

Option / Flag	Meaning
N	Seconds
Ns, Nm, Nh, Nd	Seconds, minutes, hours, days
Command &	Run it in the background

Example

```
$ sleep 30
$ sleep 30 &
$ gnome-calculator &
```

- sleep 30 blocks the terminal for 30 seconds; sleep 30 & returns the prompt at once with [1] 2541.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ sleep 30
(no prompt for 30 seconds - the terminal waits)
nasteha@ubuntu:~$ sleep 30 &
[1] 2541
nasteha@ubuntu:~$ gnome-calculator &
[2] 2547
```

Screenshot: sleep in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (Figure for foreground vs background). [1] is the job number, 2541 the PID.

4.1.7 Ctrl+C and Ctrl+Z

Purpose: Keys that act on the foreground job: Ctrl+C interrupts (SIGINT); Ctrl+Z pauses (SIGTSTP).

Syntax

```
Ctrl+C  |  Ctrl+Z
```

Options / Flags

Option / Flag	Meaning
Ctrl+C	Sends SIGINT; the job ends
Ctrl+Z	Sends SIGTSTP; the job is Stopped and waits in the job list
q	Clean way out of top

Example

```
$ sleep 100
$ sleep 100
$ top
```

- A stopped job keeps its PID.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ sleep 100
^C
nasteha@ubuntu:~$ sleep 100
^Z
[1]+  Stopped                 sleep 100
nasteha@ubuntu:~$ top
^Z
[2]+  Stopped                 top
```

Screenshot: Ctrl+C and Ctrl+Z in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: keyboard signals need an interactive terminal.

4.1.8 jobs

Purpose: Lists the jobs of this shell.

Syntax

```
jobs [-l]
```

Options / Flags

Option / Flag	Meaning
-l	Also show each job's PID
%N	Job number
+	Current job (default for bg/fg)

Example

```
$ jobs -l
```

- Jobs belong to one terminal; another terminal's list is empty.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ jobs -l
[1]-  2541 Stopped                 sleep 100
[2]+  2549 Stopped                 top
```

Screenshot: jobs in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (Figure 4.1 slide 13).

4.1.9 bg

Purpose: Resumes a stopped job in the background.

Syntax

```
bg [%N]
```

Options / Flags

Option / Flag	Meaning
%N	Job number from jobs
(no %N)	The current job (+)

Example

```
$ bg %1
```

- bg %1 carries the paused sleep on in the background.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ bg %1
```

```
[1]- sleep 100 &
```

Screenshot: bg in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output.

4.1.10 fg

Purpose: Brings a job back to the foreground.

Syntax

```
fg [%N]
```

Options / Flags

Option / Flag	Meaning
%N	Job number
(no %N)	The current job

Example

```
$ fg %2
$ fg
```

- fg %2 returns top to the screen (q to quit); fg alone takes the current job; Ctrl+C ends it.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ fg %2
[top fills the screen - press q]
nasteha@ubuntu:~$ fg
sleep 100
^C
```

Screenshot: fg in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output.

4.1.11 kill -l

Purpose: Lists signals or converts a signal number to its name.

Syntax

```
kill -l [NUMBER]
```

Options / Flags

Option / Flag	Meaning
-l	List all signal names
-l 15	Convert a number to its name

Example

```
$ kill -l | head -n 3
```

```
$ kill -l 15
$ kill -l 9
```

- Four signals matter: 1 SIGHUP (reload), 2 SIGINT (Ctrl+C), 9 SIGKILL (end now, cannot clean up), 15 SIGTERM (polite request, default).

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ kill -l | head -n 3
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
nasteha@ubuntu:~$ kill -l 15
TERM
nasteha@ubuntu:~$ kill -l 9
KILL
```

Screenshot: kill -l in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.2 Signals, kill, and Priority

A signal is a short message the kernel delivers to a process. Ask with TERM first, force with KILL last.

- Order: 1 kill PID (ask); 2 ps -p PID (wait and check); 3 kill -9 PID (last resort - unsaved work is lost, no lock-file cleanup). Check the PID with ps -p before pressing Enter.
- killall matches the exact name; pkill matches part of the name; look first with pgrep -a NAME.
- Niceness runs from -20 (greedier, administrator only) to 19 (kinder, any user); the default is 0. Any user can raise niceness of own processes; lowering it needs sudo.

No.	Name	How sent	Meaning
1	SIGHUP	kill -HUP PID / terminal closes	Hang up; many services reload configuration
2	SIGINT	Ctrl+C	Interrupt from keyboard
9	SIGKILL	kill -9 PID	End now; cannot save or clean up
15	SIGTERM	kill PID (default)	Please finish and exit

4.2.1 kill

Purpose: Sends a signal to a process by PID (or job number). Default is SIGTERM.

Syntax

```
kill [-SIGNAL] PID
kill %N
```

Options / Flags

Option / Flag	Meaning
(none)	SIGTERM (15)
-TERM	Same, written out

Option / Flag	Meaning
-HUP	Hang up / reload
-9, -KILL	Force end
%N	A job number
sudo kill	For processes you do not own

Example

```
$ sleep 500 &
$ kill 2610
$ kill -TERM 2612
$ kill %1
$ sudo kill -HUP 905
$ kill 2650
$ ps -p 2650
$ kill -9 2650
```

- Start a background sleep, stop it with TERM, confirm; then the 3-step TERM-first order.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sleep 500 &
[1] 2610
nasteha@ubuntu:~$ kill 2610
[1]+  Terminated              sleep 500
nasteha@ubuntu:~$ kill -TERM 2612
nasteha@ubuntu:~$ kill %1
nasteha@ubuntu:~$ sudo kill -HUP 905
nasteha@ubuntu:~$ kill 2650
nasteha@ubuntu:~$ ps -p 2650
    PID TTY          TIME CMD
   2650 pts/0    00:00:03 python3
nasteha@ubuntu:~$ kill -9 2650
```

Screenshot: kill in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.2 pgrep

Purpose: Finds process IDs by name; use it before pkill or killall.

Syntax

```
pgrep [-a] NAME
```

Options / Flags

Option / Flag	Meaning
-a	Show PID and full command
-u USER	Only that user's
-l	PID and name
-c	Count matches

Example

```
$ pgrep -a sleep
```

- Look before you act: two sleep processes match.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ pgrep -a sleep
2701 sleep 400
2705 sleep 900
```

Screenshot: pgrep in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.3 pkill

Purpose: Sends a signal to processes matched by (part of) their name.

Syntax

```
pkill [-u USER] [-SIGNAL] NAME
```

Options / Flags

Option / Flag	Meaning
-u USER	Only that user's processes
-9	Force
-f	Match the full command line
-x	Exact name

Example

```
$ pgrep -a sleep
$ pkill -u nasteha sleep
```

- pkill -u nasteha sleep sends TERM to every sleep you own.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ pgrep -a sleep
2701 sleep 400
2705 sleep 900
nasteha@ubuntu:~$ pkill -u nasteha sleep
```

Screenshot: pkill in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.4 killall

Purpose: Sends a signal to all processes with an exact name; reports when none is found.

Syntax

```
killall [-SIGNAL] NAME
```

Options / Flags

Option / Flag	Meaning
-u USER	Only that user's
-9	Force
-i	Ask before killing
-v	Report

Example

```
$ killall sleep
```

- killall sleep with nothing running prints 'sleep: no process found'.

Terminal (screenshot of the command and its output)



Screenshot: killall in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: killall (psmisc package) is not installed in the build sandbox.

4.2.5 Live example: start, find, stop

Purpose: One complete run: start a background sleep, find it three ways, end it with TERM.

Syntax

```
sleep 600 &  ->  jobs / pgrep / ps  ->  kill PID  ->  jobs
```

Options / Flags

Option / Flag	Meaning
jobs -l	Job list with PID
pgrep -a sleep	PID and command
ps -p PID -o pid,ppid,stat,cmd	Chosen columns (STAT S = sleeping)

Example

```
$ sleep 600 &
$ jobs -l
$ pgrep -a sleep
$ ps -p 3120 -o pid,ppid,stat,cmd
$ kill 3120
$ jobs
```

```
$ ps -p 3120
```

- TERM is enough; no -9 needed. The PID is gone afterwards.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sleep 600 &
[1] 3120
nasteha@ubuntu:~$ jobs -l
[1]+  3120 Running         sleep 600 &
nasteha@ubuntu:~$ pgrep -a sleep
3120 sleep 600
nasteha@ubuntu:~$ ps -p 3120 -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   3120    2318 S    sleep 600
nasteha@ubuntu:~$ kill 3120
[1]+  Terminated       sleep 600
nasteha@ubuntu:~$ jobs
nasteha@ubuntu:~$ ps -p 3120
    PID TTY          TIME CMD
```

Screenshot: Live example: start, find, stop in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.6 nice

Purpose: Starts a command with a chosen niceness; with no arguments prints the current niceness.

Syntax

```
nice [-n N] [COMMAND]
```

Options / Flags

Option / Flag	Meaning
(none)	Print current niceness
-n N	Niceness to start with (-20..19)

Example

```
$ nice
$ nice -n 10 sleep 300 &
$ ps -o pid,ni,comm -p 3301
```

- nice prints 0; nice -n 10 sleep 300 & starts a low-priority job; ps -o pid,ni,comm confirms NI 10.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ nice
0
nasteha@ubuntu:~$ nice -n 10 sleep 300 &
[1] 3301
nasteha@ubuntu:~$ ps -o pid,ni,comm -p 3301
    PID  NI COMMAND
   3301  10 sleep
```

Screenshot: nice in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.7 renice

Purpose: Changes the niceness of a running process.

Syntax

```
renice -n N -p PID
```

Options / Flags

Option / Flag	Meaning
-n N	New niceness
-p PID	Which process
top: r	Renice from inside top

Example

```
$ renice -n 15 -p 3301
$ renice -n 5 -p 3301
$ sudo renice -n 5 -p 3301
$ top -p 3301
```

- A normal user can only make a process kinder (higher number); lowering needs sudo.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ renice -n 15 -p 3301
3301 (process ID) old priority 10, new priority 15
nasteha@ubuntu:~$ renice -n 5 -p 3301
renice: failed to set priority for 3301 (process ID):
Permission denied
nasteha@ubuntu:~$ sudo renice -n 5 -p 3301
3301 (process ID) old priority 15, new priority 5
nasteha@ubuntu:~$ top -p 3301
```

Screenshot: renice in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.2.8 sudo (kill, renice)

Purpose: Administrator versions of the signal and priority commands.

Syntax

```
sudo COMMAND
```

Options / Flags

Option / Flag	Meaning
sudo kill -HUP 905	Tell rsyslogd to reopen its log files

Option / Flag	Meaning
sudo renice -n 5 -p PID	Make a process greedier

Example

```
$ sudo renice -n 5 -p 3301
$ sudo kill -HUP 905
```

- The administrator can lower niceness that a normal user cannot.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo renice -n 5 -p 3301
3301 (process ID) old priority 15, new priority 5
nasteha@ubuntu:~$ sudo kill -HUP 905
```

Screenshot: sudo (kill, renice) in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: the build sandbox has no sudo.

4.3 System Information

Before fixing anything, gather facts. Many tools simply read files under /proc and /sys.

- uptime reads /proc/uptime and /proc/loadavg; free reads /proc/meminfo; lscpu and nproc read /proc/cpuinfo; ps and top read /proc/<pid>/.
- Load average = processes running or waiting (1, 5, 15 minutes); compare with nproc: below it means spare capacity.
- free: Linux uses spare RAM as disk cache; read the available column, not free.
- /proc is not a folder on disk: files show size 0 but are full of text, generated when read; numbered folders are PIDs. /sys is the kernel's map of hardware and drivers (block, class, class/net).

4.3.1 uname (continued: -m, -n, -a)

Purpose: Prints kernel and machine facts (continued from Chapter 1).

Syntax

```
uname [-r] [-m] [-n] [-a]
```

Options / Flags

Option / Flag	Meaning
-r	Kernel release
-m	Machine type (x86_64; Raspberry Pi: aarch64)
-n	Hostname
-a	All fields on one line
-s	Kernel name

Example

```
$ uname -r
```

```
$ uname -m
$ uname -n
$ uname -a
```

- Quote the kernel release when you report a problem; uname -a is ready to paste into a lab report.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ uname -r
6.18.44-fc-v378
nasteha@ubuntu:~$ uname -m
x86_64
nasteha@ubuntu:~$ uname -n
ubuntu
nasteha@ubuntu:~$ uname -a
Linux ubuntu 6.18.44-fc-v378 #1 SMP PREEMPT_DYNAMIC @0 x86_64 x86_64 x86_64 GNU/Linux
```

Screenshot: uname (continued: -m, -n, -a) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.2 hostnamectl

Purpose: Shows hostname, OS, kernel and whether the machine is virtual.

Syntax

```
hostnamectl [hostname]
```

Options / Flags

Option / Flag	Meaning
(none)	Full summary
hostname	Print only the hostname
set-hostname NAME	Change it (needs sudo; not used this week)

Example

```
$ hostnamectl
$ hostnamectl hostname
```

- Virtualization oracle and VirtualBox mean a virtual machine.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ hostnamectl
 Static hostname: ubuntu
       Chassis: vm
  Virtualization: oracle
Operating System: Ubuntu 24.04.3 LTS
        Kernel: Linux 6.8.0-85-generic
    Architecture: x86-64
  Hardware Model: VirtualBox
nasteha@ubuntu:~$ hostnamectl hostname
ubuntu
```

Screenshot: hostnamectl in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: hostnamectl needs systemd, which is not running in the build sandbox.

4.3.3 uptime

Purpose: Shows time, uptime, users and the load average.

Syntax

```
uptime [-p] [-s]
```

Options / Flags

Option / Flag	Meaning
-p	Uptime in words
-s	When the system booted
(none)	One-line summary

Example

```
$ uptime
$ uptime -p
$ uptime -s
```

- Load for 1, 5 and 15 minutes; near zero means idle.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ uptime
 18:43:52 up 1 min,  0 user,  load average: 0.41, 0.15, 0.05
nasteha@ubuntu:~$ uptime -p
up 1 minute
nasteha@ubuntu:~$ uptime -s
2026-11-11 18:42:04
```

Screenshot: uptime in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.4 nproc

Purpose: Prints the number of processing units (CPUs) available.

Syntax

```
nproc [--all]
```

Options / Flags

Option / Flag	Meaning
--all	Count all installed processors
(none)	Available to this process

Example

```
$ nproc
```

- Compare the count with the load average.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ nproc
1
```

Screenshot: nproc in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.5 free

Purpose: Reports RAM and swap use.

Syntax

```
free [-h] [-m] [-s N]
```

Options / Flags

Option / Flag	Meaning
-h	Human units (Mi, Gi)
-m	Mebibytes
-s N	Repeat every N seconds (Ctrl+C stops)
-t	Show a total line

Example

```
$ free -h
$ free -m
```

- Read the available column, not free.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.9Gi       274Mi       3.5Gi        16Mi       328Mi       3.6Gi
Swap:             0B          0B          0B
nasteha@ubuntu:~$ free -m
               total        used        free      shared  buff/cache   available
Mem:            3990         274        3604          16         328        3715
Swap:              0           0           0
```

Screenshot: free in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.6 lscpu

Purpose: Summarizes the processor: architecture, CPUs, model, hypervisor.

Syntax

```
lscpu
```

Options / Flags

Option / Flag	Meaning
(none)	Full summary
| less	Page the list

Example

```
$ lscpu | head -n 12
```

- A VM shows the CPUs given in VirtualBox, not every host core.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ lscpu | head -n 12
Architecture:                    x86_64
CPU op-mode(s):                  32-bit, 64-bit
Address sizes:                   52 bits physical, 57 bits virtual
Byte Order:                      Little Endian
CPU(s):                          1
On-line CPU(s) list:             0
Vendor ID:                       GenuineIntel
Model name:                      Intel(R) Xeon(R) Processor @ 2.10GHz
CPU family:                      6
Model:                           207
Thread(s) per core:              1
Core(s) per socket:              1
```

Screenshot: lscpu in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.7 lsblk

Purpose: Lists block devices: disks, partitions, CD drive.

Syntax

```
lsblk [options]
```

Options / Flags

Option / Flag	Meaning
-e 7	Hide loop devices
-d	Whole disks only
-o COLS	Choose columns (NAME,SIZE,MODEL)
-f	Show filesystems

Example

```
$ lsblk -e 7
$ lsblk -e 7 -d -o NAME,SIZE,MODEL
```

- The VM sees a virtual disk sda and a virtual CD drive sr0.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ lsblk -e 7
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   25G  0 disk
├─sda1   8:1    0    1M  0 part
└─sda2   8:2    0   25G  0 part /
sr0     11:0    1 1024M  0 rom
nasteha@ubuntu:~$ lsblk -e 7 -d -o NAME,SIZE,MODEL
NAME  SIZE MODEL
sda    25G VBOX HARDDISK
sr0  1024M VBOX CD-ROM
```

Screenshot: lsblk in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (values from the course slides; PIDs and times vary). The build sandbox is a minimal container with its own system processes and disks, so a clean transcript is shown instead.

4.3.8 /proc files

Purpose: The kernel's live window: read files such as uptime, loadavg, meminfo, cpuinfo and per-PID folders.

Syntax

```
ls /proc
cat /proc/FILE
```

Options / Flags

Option / Flag	Meaning
/proc/uptime	Seconds since boot
/proc/loadavg	The three load averages
/proc/meminfo	Memory (kB)
/proc/cpuinfo	CPU details
/proc/PID/status	Name, state, PID, PPID
/proc/PID/comm	Short command name

Example

```
$ ls /proc | head -n 12
$ ls -l /proc/meminfo
$ cat /proc/uptime
$ cat /proc/loadavg
$ ls /proc/$$ | head -n 8
$ head -n 7 /proc/$$/status
$ cat /proc/1/comm
$ grep -m 1 "model name" /proc/cpuinfo
$ grep -c ^processor /proc/cpuinfo
$ head -n 3 /proc/meminfo
```

- ls -l /proc/meminfo shows size 0 although it is full of text.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls /proc | head -n 12
1
```

```
10
11
12
1246
1250
13
14
15
16
17
18
nasteha@ubuntu:~$ ls -l /proc/meminfo
-r--r--r-- 1 root root 0 Nov 11 18:42 /proc/meminfo
nasteha@ubuntu:~$ cat /proc/uptime
107.69 50.36
nasteha@ubuntu:~$ cat /proc/loadavg
0.41 0.15 0.05 2/87 2572
nasteha@ubuntu:~$ ls /proc/$$ | head -n 8
arch_status
attr
autogroup
auxv
cgroup
clear_refs
cmdline
comm
nasteha@ubuntu:~$ head -n 7 /proc/$$/status
Name:   bash
Umask:  0022
State:  S (sleeping)
Tgid:   2566
Ngid:   0
Pid:    2566
PPid:   2565
nasteha@ubuntu:~$ cat /proc/1/comm
process_api
nasteha@ubuntu:~$ grep -m 1 "model name" /proc/cpuinfo
model name      : Intel(R) Xeon(R) Processor @ 2.10GHz
nasteha@ubuntu:~$ grep -c ^processor /proc/cpuinfo
1
nasteha@ubuntu:~$ head -n 3 /proc/meminfo
MemTotal:        4086428 kB
MemFree:         3691144 kB
MemAvailable:    3805240 kB
```

Screenshot: /proc files in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.3.9 /sys files

Purpose: Devices and drivers as files; read only.

Syntax

```
ls /sys/CATEGORY
cat /sys/.../FILE
```

Options / Flags

Option / Flag	Meaning
block	Disks as lsblk sees them
class	Devices grouped by type
class/net	One folder per network card
.../address	The card's hardware address

Example

```
$ ls /sys
$ ls /sys/block
$ ls /sys/class/net
$ cat /sys/class/net/lo/address
```

- Interface names such as enp0s3 vary by machine.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ls /sys
block bus class dev devices firmware fs kernel module power
nasteha@ubuntu:~$ ls /sys/block
loop0 loop2 loop4 loop6 vda vdc zram0
loop1 loop3 loop5 loop7 vdb vdd
nasteha@ubuntu:~$ ls /sys/class/net
eth0 ifb0 ifb1 lo
nasteha@ubuntu:~$ cat /sys/class/net/lo/address
00:00:00:00:00:00
```

Screenshot: /sys files in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.4 Services with systemd

A service is a background program. systemd (PID 1) starts and watches them; each is described by a unit file ending in .service. We control them with systemctl.

- stop/start act now; enable/disable act at the next boot; disable --now = end and do not start; enable --now = start and start at boot.
- start/stop/restart/reload need sudo. restart = new PID and a brief drop; reload re-reads configuration without stopping (not every service supports it; reload-or-restart picks automatically).
- WARNING: do not stop or disable ssh on a machine you reach only over SSH. Keep a console (the VirtualBox window).

Command	Right now	At next boot
stop	Ends	Starts again if enabled
disable	Keeps running	Does not start
disable --now	Ends	Does not start
enable --now	Starts	Starts

4.4.1 systemctl cat

Purpose: Prints a service's unit file (read it, do not edit it).

Syntax

```
systemctl cat UNIT
```

Options / Flags

Option / Flag	Meaning
cat UNIT	Show the unit file
ExecStart=	What systemd runs

Example

```
$ ps -p 1 -o pid,comm
$ systemctl cat cron
```

- systemd is PID 1; ps -p 1 -o pid,comm shows it.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ ps -p 1 -o pid,comm
  PID COMMAND
    1 systemd
nasteha@ubuntu:~$ systemctl cat cron
# /usr/lib/systemd/system/cron.service
[Unit]
Description=Regular background program processing daemon
[Service]
ExecStart=/usr/sbin/cron -f -P $EXTRA_OPTS
```

Screenshot: systemctl cat in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: systemd is not running in the build sandbox (PID 1 is not systemd), so all systemctl transcripts follow the course slides.

4.4.2 systemctl status

Purpose: First question for any service: enabled? running? last log lines?

Syntax

```
systemctl status UNIT
```

Options / Flags

Option / Flag	Meaning
status	Loaded/enabled, Active, Main PID, last log lines
is-active	Prints active or inactive
is-enabled	Prints enabled or disabled

Example

```
$ systemctl status cron
$ systemctl is-active cron
```

```
$ systemctl is-enabled cron
```

- Active: active (running) = up; red failed = trouble.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ systemctl status cron
● cron.service - Regular background program processing daemon
     Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-10-15 08:02:11 EAT; 2h 41min ago
   Main PID: 812 (cron)
Oct 15 10:17:01 ubuntu CRON[3051]: (root) CMD (cd / && run-parts --report /etc/cron.hourly)
nasteha@ubuntu:~$ systemctl is-active cron
active
nasteha@ubuntu:~$ systemctl is-enabled cron
enabled
```

Screenshot: systemctl status in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.4.3 systemctl start / stop / restart

Purpose: Change what a service is doing right now (administrator rights, so sudo).

Syntax

```
sudo systemctl ACTION UNIT
```

Options / Flags

Option / Flag	Meaning
start	Start now
stop	Stop now
restart	Stop, then start: a new PID
show -p MainPID UNIT	Print the main PID

Example

```
$ sudo systemctl stop cron
$ systemctl is-active cron
$ sudo systemctl start cron
$ systemctl is-active cron
$ sudo systemctl restart cron
$ systemctl show -p MainPID cron
```

- No output from start or stop means success; check with is-active. Restart gave a new process: 812 became 3388.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo systemctl stop cron
[sudo] password for nasteha:
nasteha@ubuntu:~$ systemctl is-active cron
inactive
```

```
nasteha@ubuntu:~$ sudo systemctl start cron
nasteha@ubuntu:~$ systemctl is-active cron
active
nasteha@ubuntu:~$ sudo systemctl restart cron
nasteha@ubuntu:~$ systemctl show -p MainPID cron
MainPID=3388
```

Screenshot: systemctl start / stop / restart in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.4.4 systemctl reload

Purpose: Asks a running service to re-read its configuration without stopping it.

Syntax

```
sudo systemctl reload UNIT
sudo systemctl reload-or-restart UNIT
```

Options / Flags

Option / Flag	Meaning
reload	Re-read config; only if supported
reload-or-restart	Reload if supported, otherwise restart
restart	Works for every service

Example

```
$ sudo systemctl reload ssh
$ sudo systemctl reload cron
$ sudo systemctl reload-or-restart cron
```

- cron has no reload action; use restart or reload-or-restart.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo systemctl reload ssh
nasteha@ubuntu:~$ sudo systemctl reload cron
Failed to reload cron.service: Job type reload is
not applicable for unit cron.service.
nasteha@ubuntu:~$ sudo systemctl reload-or-restart cron
```

Screenshot: systemctl reload in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.4.5 systemctl enable / disable

Purpose: Decide whether a service starts at the next boot; --now also acts immediately.

Syntax

```
sudo systemctl enable [--now] UNIT
sudo systemctl disable [--now] UNIT
```

Options / Flags

Option / Flag	Meaning
enable	Start at every boot
disable	Do not start at boot
--now	Also start or stop immediately

Example

```
$ sudo systemctl disable cron
$ systemctl is-enabled cron
$ sudo systemctl enable --now cron
$ systemctl is-active cron
$ systemctl is-enabled cron
```

- disable leaves cron running until you stop it or reboot.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo systemctl disable cron
Removed "/etc/systemd/system/multi-user.target.wants/cron.service".
nasteha@ubuntu:~$ systemctl is-enabled cron
disabled
nasteha@ubuntu:~$ sudo systemctl enable --now cron
Created symlink /etc/systemd/system/multi-user.target.wants/cron.service → ...
nasteha@ubuntu:~$ systemctl is-active cron
active
nasteha@ubuntu:~$ systemctl is-enabled cron
enabled
```

Screenshot: systemctl enable / disable in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide). WARNING: never stop or disable ssh on a remote-only machine.

4.4.6 systemctl list-units

Purpose: Shows what systemd has loaded; filter by type and state.

Syntax

```
systemctl list-units --type=service --state=STATE
```

Options / Flags

Option / Flag	Meaning
--type=service	Only services
--state=running	Running ones
--state=failed	Failed ones (first lead when troubleshooting)

Example

```
$ systemctl list-units --type=service --state=running
$ systemctl list-units --type=service --state=failed
```

- Zero failed is a good sign.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ systemctl list-units --type=service --state=running
  UNIT                    LOAD   ACTIVE SUB     DESCRIPTION
  cron.service              loaded active running Regular background program processing daemon
  rsyslog.service           loaded active running System Logging Service
  systemd-journald.service  loaded active running Journal Service
nasteha@ubuntu:~$ systemctl list-units --type=service --state=failed
0 loaded units listed.
```

Screenshot: systemctl list-units in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide); the list on your VM is longer.

4.5 Logs, Boot, and First Troubleshooting

When something fails Linux has usually written down why: the systemd journal, text logs in /var/log and kernel messages.

- /var/log files: syslog (general), auth.log (logins, sudo, SSH), kern.log (kernel), journal/ (binary journal files).
- Service misbehaves: 1 systemctl status ssh; 2 journalctl -u ssh (find the first error); 3 /etc/ssh/ and sudo sshd -t (no output = OK).
- Machine is slow: 1 uptime (load vs nproc); 2 free -h (available memory); 3 top (press M or P, note the PID) then wait, renice, or kill with TERM.
- Shut down from inside Linux; in VirtualBox 'Power off the machine' is pulling the plug. Take a VM snapshot before any boot experiment.

4.5.1 journalctl

Purpose: Reads the systemd journal (kernel and all services).

Syntax

```
journalctl [options]
```

Options / Flags

Option / Flag	Meaning
-n N	Last N entries
-b	This boot only
-p err	Errors and worse
-u UNIT	One service only
--since "1 hour ago"	From a start time
-e	Jump to the end
-x	Add explanation text
-f	Follow new lines (Ctrl+C stops)
-k	Kernel messages only

Example

```
$ journalctl -n 3
$ journalctl -b -p err
$ journalctl -u cron -n 4
$ journalctl -u cron --since "1 hour ago"
$ journalctl -u ssh -n 20
```

- Entries show time, host, program[PID] and message. 'Insufficient permissions' needs sudo.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ journalctl -n 3
Oct 15 10:40:01 ubuntu CRON[3290]: pam_unix(cron:session): session opened
Oct 15 10:40:01 ubuntu CRON[3291]: (root) CMD (command -v debian-sa1 ...)
Oct 15 10:40:01 ubuntu CRON[3290]: pam_unix(cron:session): session closed
nasteha@ubuntu:~$ journalctl -b -p err
nasteha@ubuntu:~$ journalctl -u cron -n 4
Oct 15 10:31:12 ubuntu systemd[1]: Stopping cron.service - Regular b...
Oct 15 10:31:12 ubuntu systemd[1]: cron.service: Deactivated successfully.
Oct 15 10:31:12 ubuntu systemd[1]: Stopped cron.service - Regular b...
Oct 15 10:31:20 ubuntu systemd[1]: Started cron.service - Regular b...
nasteha@ubuntu:~$ journalctl -u cron --since "1 hour ago"
nasteha@ubuntu:~$ journalctl -u ssh -n 20
```

Screenshot: journalctl in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slides): the build sandbox has no systemd journal.

4.5.2 journalctl -xe and -f

Purpose: -xe jumps to the end with explanations; -f follows the journal live.

Syntax

```
journalctl -xe
journalctl [-u UNIT] -f
```

Options / Flags

Option / Flag	Meaning
-e	Jump to the end
-x	Explanatory text
-f	Follow; Ctrl+C stops
-u UNIT -f	Follow one service

Example

```
$ journalctl -xe
$ journalctl -f
$ journalctl -u cron -f
```

- Leave journalctl -f open in a first terminal; logger in a second one makes a line appear at once.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ journalctl -xe
nasteha@ubuntu:~$ journalctl -f
Oct 15 10:52:30 ubuntu nasteha[3415]: week 8 test from nasteha
nasteha@ubuntu:~$ journalctl -u cron -f
```

Screenshot: journalctl -xe and -f in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.5.3 logger

Purpose: Writes a test message to the system log.

Syntax

```
logger [OPTION]... MESSAGE
```

Options / Flags

Option / Flag	Meaning
-t TAG	Tag the message
-p PRIORITY	Facility.level

Example

```
$ logger "week 8 test from nasteha"
```

- Run logger in a second terminal while journalctl -f is open in the first.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ logger "week 8 test from nasteha"
```

Screenshot: logger in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative: the message appears in journalctl -f as 'Oct 15 10:52:30 ubuntu nasteha[3415]: week 8 test from nasteha'.

4.5.4 /var/log (less, tail, ls)

Purpose: Classic text logs written by rsyslog next to the journal; read with the tools you know.

Syntax

```
less /var/log/FILE
tail -f /var/log/FILE
```

Options / Flags

Option / Flag	Meaning
ls /var/log	List logs
less FILE	Page (/ search, q quit)
tail -n 3 FILE	Newest lines
tail -f FILE	Follow, like journalctl -f

Example

```
$ ls /var/log | head -n 8
$ tail -n 3 /var/log/dpkg.log
```

- syslog = general; auth.log = logins and sudo; kern.log = kernel.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ ls /var/log | head -n 8
README
alternatives.log
apt
bootstrap.log
btmp
dpkg.log
faillog
fontconfig.log
nasteha@ubuntu:~$ tail -n 3 /var/log/dpkg.log
2026-10-30 01:07:39 trigproc ca-certificates-java:all 20240118 <none>
2026-10-30 01:07:39 status half-configured ca-certificates-java:all 20240118
2026-10-30 01:07:40 status installed ca-certificates-java:all 20240118
```

Screenshot: /var/log (less, tail, ls) in the Ubuntu terminal - **REAL OUTPUT (executed on Ubuntu 24.04)**

4.5.5 auth.log, journalctl _COMM=sudo, last

Purpose: Every sudo command and login is recorded.

Syntax

```
grep sudo /var/log/auth.log
journalctl _COMM=sudo
last -n 2
```

Options / Flags

Option / Flag	Meaning
grep sudo /var/log/auth.log	Who ran sudo, from where, what command
journalctl _COMM=sudo -n 1	Same from the journal
last -n 2	Logins and reboots (from /var/log/wtmp)

Example

```
$ grep sudo /var/log/auth.log | tail -n 1
$ journalctl _COMM=sudo -n 1
$ last -n 2
```

- Read before you restart.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ grep sudo /var/log/auth.log | tail -n 1
2026-10-15T10:31:12.402118+03:00 ubuntu sudo:  nasteha : TTY=pts/0 ; PWD=/home/nasteha ; USER=root ;
COMMAND=/usr/bin/systemctl stop cron
```

```
nasteha@ubuntu:~$ journalctl _COMM=sudo -n 1
Oct 15 10:31:12 ubuntu sudo[3360]:  nasteha : TTY=pts/0 ; PWD=/home/nasteha ; USER=root ;
COMMAND=/usr/bin/systemctl stop cron
nasteha@ubuntu:~$ last -n 2
nasteha  tty2         tty2             Thu Oct 15 08:03   still logged in
reboot   system boot  6.8.0-85-generic Thu Oct 15 08:02   still running
```

Screenshot: auth.log, journalctl _COMM=sudo, last in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.5.6 dmesg

Purpose: Prints kernel messages (hardware found, drivers loaded). Ubuntu limits it to administrators.

Syntax

```
sudo dmesg [-T]
journalctl -k
```

Options / Flags

Option / Flag	Meaning
-T	Clock times instead of seconds
-k (journalctl)	Kernel messages only
-b	This boot only
| less	Page from the first message

Example

```
$ dmesg
$ sudo dmesg -T | tail -n 2
$ sudo dmesg | less
$ journalctl -k -b
```

- A normal user gets 'Operation not permitted'; add sudo.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~

nasteha@ubuntu:~$ dmesg
dmesg: read kernel buffer failed: Operation not permitted
nasteha@ubuntu:~$ sudo dmesg -T | tail -n 2
[Thu Oct 15 08:02:14 2026] sd 2:0:0:0: [sda] Attached SCSI disk
[Thu Oct 15 08:02:19 2026] e1000 0000:00:03.0 enp0s3: NIC Link is Up
nasteha@ubuntu:~$ sudo dmesg | less
nasteha@ubuntu:~$ journalctl -k -b
```

Screenshot: dmesg in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.5.7 shutdown

Purpose: Schedules or performs a clean shutdown or reboot.

Syntax

```
sudo shutdown [-r] TIME [MESSAGE]
```

Options / Flags

Option / Flag	Meaning
now	Immediately
+N	In N minutes
-r	Reboot instead of power off
-c	Cancel a planned shutdown

Example

```
$ sudo shutdown -r +5 "Rebooting for the lab"
$ sudo shutdown -c
```

- Shut down from inside Linux so services stop in order.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo shutdown -r +5 "Rebooting for the lab"
Reboot scheduled for Thu 2026-10-15 11:05:00 EAT,
use 'shutdown -c' to cancel.
nasteha@ubuntu:~$ sudo shutdown -c
```

Screenshot: shutdown in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output: not executed - a shutdown would stop the machine (a reboot experiment). Take a VM snapshot first.

4.5.8 reboot

Purpose: Restarts the machine now.

Syntax

```
sudo reboot
```

Options / Flags

Option / Flag	Meaning
(none)	Restart now
--force	Immediate, without stopping services (avoid)

Example

```
$ sudo reboot
```

- Restart now.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

Screenshot: reboot in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative: not executed (would restart the machine).

4.5.9 poweroff

Purpose: Stops every service and switches the machine off now.

Syntax

```
sudo poweroff
```

Options / Flags

Option / Flag	Meaning
(none)	Power off now

Example

```
$ sudo poweroff
```

- Never use 'Power off the machine' of VirtualBox as a normal shutdown.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ sudo poweroff
```

Screenshot: poweroff in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative: not executed (would power off the machine).

4.5.10 sshd -t

Purpose: Checks sshd_config for mistakes before restarting ssh (step 3 of the service order).

Syntax

```
sudo sshd -t
```

Options / Flags

Option / Flag	Meaning
-t	Test configuration and exit; no output means OK
ls /etc/ssh	moduli ssh_config ssh_config.d sshd_config sshd_config.d

Example

```
$ systemctl status ssh
$ journalctl -u ssh -n 30
$ ls /etc/ssh
$ sudo sshd -t
```

- Order: systemctl status ssh > journalctl -u ssh -n 30 > ls /etc/ssh > sudo sshd -t.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
```

```
nasteha@ubuntu:~$ systemctl status ssh
nasteha@ubuntu:~$ journalctl -u ssh -n 30
nasteha@ubuntu:~$ ls /etc/ssh
moduli  ssh_config  ssh_config.d  sshd_config  sshd_config.d  ...
nasteha@ubuntu:~$ sudo sshd -t
```

Screenshot: sshd -t in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).

4.5.11 Slow-machine order: uptime, free -h, top

Purpose: Three commands from this chapter in order narrow the question before you act.

Syntax

```
uptime ; free -h ; top
```

Options / Flags

Option / Flag	Meaning
uptime	Load against nproc; rising?
free -h	Little available? swap in use?
top	Press P or M; note the PID

Example

```
$ uptime
$ free -h
$ top
```

- Slide example: load 2.31 on a 2-CPU VM and only 180Mi available: memory is short.

Terminal (screenshot of the command and its output)

```
nasteha@ubuntu: ~
nasteha@ubuntu:~$ uptime
 11:02:44 up  3:00,  1 user,  load average: 2.31, 1.40, 0.62
nasteha@ubuntu:~$ free -h
Mem:          3.8Gi       3.5Gi       112Mi        40Mi       310Mi       180Mi
nasteha@ubuntu:~$ top
```

Screenshot: Slow-machine order: uptime, free -h, top in the Ubuntu terminal - **REPRESENTATIVE OUTPUT (not executed here)**

Note: Representative output (course slide).